[College logo] *[University logo]*

A MAJOR PROJECT-I REPORT

on

IndexR: A Vectorless Research Assistant with Section- and Page-Level Cited Answers

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science & Engineering (AI&ML)

CSEE-712P Major Project-I · 7th Semester, CSE (AI&ML)

Submitted by:

Name	Roll No.
Aman	240104020002
Hemlata	240104020003
Isaaq Mohammad	240104020004
Jatin Kharwal	2040104020005
Nitika Sharma	240104020007

Under the guidance of:

Er. Shivani Thakur
Assistant Professor
(Computer Engineering)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (AI & ML)

JAWAHARLAL NEHRU GOVT. ENGINEERING COLLEGE, SUNDERNAGAR

(Affiliated to Himachal Pradesh Technical University, Hamirpur, H.P.)

Session: August–December 2026

CERTIFICATE

This is to certify that the project report entitled **"IndexR: A Vectorless Research Assistant with Section- and Page-Level Cited Answers"**, submitted by Aman (240104020002), Hemlata (240104020003), Isaaq Mohammad (240104020004), Jatin Kharwal (2040104020005) and Nitika Sharma (240104020007), students of the 7th semester, Bachelor of Technology in Computer Science & Engineering (AI&ML), to the Department of Computer Science & Engineering (AI & ML), Jawaharlal Nehru Govt. Engineering College, Sundernagar, affiliated to Himachal Pradesh Technical University, Hamirpur, H.P., in partial fulfilment of the requirements of the course CSEE-712P Major Project-I, is a record of bonafide work carried out by them under my supervision and guidance during the session August–December 2026.

The matter embodied in this report is original and has not been submitted, to the best of my knowledge, for the award of any other degree or diploma.

Signature of Guide
Er. Shivani Thakur
Assistant Professor
(Computer Engineering)

Signature of Head of Department

Department of CSE (AI & ML)
JNGEC, Sundernagar

Date: _____
Place: Sundernagar

DECLARATION

We hereby declare that the project report entitled **"IndexR: A Vectorless Research Assistant with Section- and Page-Level Cited Answers"**, submitted in partial fulfilment of the requirements of the course CSEE-712P Major Project-I for the degree of Bachelor of Technology in Computer Science & Engineering (AI&ML) at Jawaharlal Nehru Govt. Engineering College, Sundernagar, is an authentic record of our own work carried out under the guidance of Er. Shivani Thakur, Assistant Professor.

We further declare that the work reported in this report has not been submitted, either in part or in full, for the award of any other degree or diploma in this or any other institute or university. All sources of information used have been duly acknowledged and cited.

No.	Name	Roll No.	Signature
1	Aman	240104020002	
2	Hemlata	240104020003	
3	Isaaq Mohammad	240104020004	
4	Jatin Kharwal	2040104020005	
5	Nitika Sharma	240104020007	

Date: _____

Place: Sundernagar

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **Er. Shivani Thakur**, Assistant Professor, for constant guidance, encouragement and valuable suggestions throughout this project. The guidance received at every stage, from defining the problem to reviewing the design and this report, shaped the work presented here.

We are thankful to the Head of the Department of Computer Science & Engineering (AI & ML) and to all faculty members of the department for providing the facilities, support and academic environment needed to complete this project. We also thank the Principal, Jawaharlal Nehru Govt. Engineering College, Sundernagar, for the opportunity to undertake this work as part of the Bachelor of Technology programme.

We acknowledge the open scholarly infrastructure that made this project possible, including arXiv, OpenAlex and Semantic Scholar, and the open-source communities behind Django, PostgreSQL, Celery, React and the many libraries we used.

Finally, we thank our families and friends for their patience, support and encouragement throughout the semester.

Project Team
Aman (240104020002)
Hemlata (240104020003)
Isaaq Mohammad (240104020004)
Jatin Kharwal (2040104020005)
Nitika Sharma (240104020007)

ABSTRACT

Researchers increasingly turn to large language models (LLMs) for help with the literature, but these models answer fluently while citing nothing, or citing sources that do not exist. Retrieval-augmented generation (RAG) addresses this by grounding answers in retrieved text, yet most RAG systems depend on dense vector embeddings and a dedicated vector database, which add cost and operational complexity, blur exact technical terms such as model names, dataset names and metric values, and usually cite at the level of whole documents. Existing tools also expect the user to find and upload every paper, and they do not notice when new work appears.

This project presents **IndexR**, a vectorless research assistant that finds the right papers for a question, answers with citations traceable to the exact section and page, and keeps each research topic up to date. Papers are discovered automatically from arXiv, OpenAlex and Semantic Scholar, ingested once into a shared corpus, and split into section-aware chunks that carry their page ranges. Retrieval uses PostgreSQL full-text search with a weighted, GIN-indexed `tsvector`, boosted by citation count and recency, broadened by LLM query expansion and refined by a cross-encoder reranker. A two-gate rule decides when the local corpus is insufficient and external discovery is needed. The answer step numbers the retrieved passages and maps every `[n]` marker in the generated answer back to its paper, section and page.

The system is implemented as a Django REST Framework backend with Celery workers, a React web application, a Model Context Protocol (MCP) server that exposes eleven research tools to AI assistants, private PDF uploads visible only to their owner, and a live-tracking module that checks tracked topics and papers daily for new papers, new citations and new arXiv versions. The implementation is validated by 416 automated tests, query-count tests that keep every list endpoint at a constant number of database queries, and live runs against the real external services. On a development corpus of 137 papers and 4,930 chunks, full-text retrieval takes a median of 2.2 ms per query, and an answer drawn from the existing corpus is produced in about 11 seconds end to end.

Keywords: Retrieval-augmented generation, lexical retrieval, full-text search, cross-encoder reranking, citation grounding, large language models, scholarly search, Model Context Protocol.

TABLE OF CONTENTS

CERTIFICATE ..... **i**

DECLARATION ..... **ii**

ACKNOWLEDGEMENT ..... **iii**

ABSTRACT ..... **iv**

LIST OF FIGURES ..... **viii**

LIST OF TABLES ..... **ix**

LIST OF ABBREVIATIONS ..... **x**

1 INTRODUCTION AND PROBLEM STATEMENT **1**

1.1 Background 1

1.2 Problem Statement 1

1.3 Motivation 2

1.4 Societal Relevance 3

1.5 Scope of the Project 3

1.6 Organisation of the Report 4

2 OBJECTIVES **5**

3 LITERATURE REVIEW **7**

3.1 Overview 7

3.2 Lexical Retrieval 7

3.3 Dense Retrieval and Vector Search 7

3.4 Neural Reranking 8

3.5 Retrieval-Augmented Generation 8

3.6 Query Expansion and Rewriting with LLMs 8

3.7 Hallucination, Attribution and Citation 9

3.8 Open Scholarly Data Infrastructure 9

3.9 Integration with AI Assistants 9

3.10 Summary of Related Work 10

3.11 Research Gap Analysis 10

4 PROPOSED METHODOLOGY **12**

4.1 Approach 12

4.2 Feasibility Study 12

4.2.1 Technical Feasibility 12

4.2.2 Economic Feasibility 12

4.2.3 Operational Feasibility 12

4.2.4 Schedule Feasibility....13
4.3 System Architecture....13
4.4 Block Diagram....14
4.5 Data Flow Diagrams....15
4.6 Use Cases....16
4.7 Working Principle....17
4.7.1 Paper Discovery....18
4.7.2 Ingestion and Section-Aware Chunking....18
4.7.3 Lexical Ranking with Metadata Boost....19
4.7.4 Query Expansion....19
4.7.5 Cross-Encoder Reranking....20
4.7.6 Two-Gate Discovery Rule....20
4.7.7 Grounded Answer Generation and Citation Mapping....20
4.7.8 Streaming the Answer....21
4.8 Private Uploads and Visibility....21
4.9 Live Tracking....22
4.10 MCP Integration....23
4.11 Database Design....25
4.12 API Design....27
4.13 Security and Access Control....28
5 TOOLS AND TECHNOLOGIES....29
5.1 Software Stack....29
5.2 External Services and APIs....29
5.3 Development Environment....30
5.4 Deployment Requirements....30
6 EXPECTED OUTCOMES AND VALIDATION....31
6.1 Deliverables....31
6.2 User Interface....31
6.3 Requirement Verification....38
6.4 Automated Testing....39
6.5 Performance and Efficiency....39
6.6 Live Functional Validation....40
6.7 Validation Metrics for Evaluation....41
6.8 Discussion and Limitations....41
7 TIMELINE....42
8 CONCLUSION AND FUTURE SCOPE....43

8.1 Conclusion..... 43
8.2 Future Scope..... 43
REFERENCES.....45

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 4.1	System architecture of IndexR	14
Figure 4.2	Block diagram of the IndexR pipeline	15
Figure 4.3	Level-0 (context) data flow diagram	15
Figure 4.4	Level-1 data flow diagram	16
Figure 4.5	Use-case diagram	17
Figure 4.6	Flowchart of answering a question	18
Figure 4.7	Ingestion and chunking pipeline	19
Figure 4.8	Sequence diagram of a streamed answer	21
Figure 4.9	Flowchart of the daily tracking check	23
Figure 4.10	Connecting an AI assistant through MCP	24
Figure 4.11	Entity–relationship diagram: users, papers and the corpus	26
Figure 4.12	Entity–relationship diagram: workspaces, chat and tracking	27
Figure 6.1	Login page	32
Figure 6.2	Home page listing workspaces, with the unseen-updates badge	32
Figure 6.3	Workspace with the papers panel and a cited answer with its sources	33
Figure 6.4	Citation preview showing the paper, section, page and passage	33
Figure 6.5	Adding a paper from the corpus to a workspace	34
Figure 6.6	Uploading a private PDF	34
Figure 6.7	Papers mentioned by the workspace's answers	35
Figure 6.8	Tracking panel with tracked topics and updates (new citations)	35
Figure 6.9	Creating an access token with client setup for MCP	36
Figure 6.10	Workspace in dark theme	36
Figure 6.11	Workspace on a mobile screen	37
Figure 6.12	IndexR tools used from an AI assistant (Claude Desktop) through MCP	37
Figure 6.13	Django admin: moderating the shared corpus	38
Figure 7.1	Gantt chart of the project schedule	42

LIST OF TABLES

Table No.	Title	Page No.
Table 1.1	Scope of the project	3
Table 2.1	Project objectives, success measures and target weeks	5
Table 3.1	Summary of reviewed literature and its relevance to IndexR	10
Table 4.1	The two-gate rule for triggering discovery	20
Table 4.2	Tools exposed by the IndexR MCP server	24
Table 4.3	Main database entities	25
Table 4.4	Main REST API endpoints	27
Table 5.1	Software stack	29
Table 5.2	External services used by IndexR	29
Table 5.3	Development and test environment	30
Table 6.1	Project deliverables	31
Table 6.2	Requirement verification matrix	38
Table 6.3	Automated tests by application	39
Table 6.4	Measured performance on the development corpus	39
Table 6.5	Live validation runs	40
Table 6.6	Validation metrics and targets	41
Table 7.1	Weekly plan with milestones	42

LIST OF ABBREVIATIONS

API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BM25	Best Matching 25 (probabilistic ranking function)
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
DOI	Digital Object Identifier
DRF	Django REST Framework
ER	Entity–Relationship
FTS	Full-Text Search
GIN	Generalized Inverted Index
HNSW	Hierarchical Navigable Small World (graph index)
HTTP(S)	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MCP	Model Context Protocol
ORM	Object–Relational Mapper
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
S2	Semantic Scholar
SHA	Secure Hash Algorithm
SPA	Single-Page Application
SSE	Server-Sent Events
UI	User Interface
UUID	Universally Unique Identifier

CHAPTER 1
INTRODUCTION AND PROBLEM STATEMENT

1.1 Background

The volume of scientific publishing has grown to a point where no researcher can keep up by reading alone. Repositories such as arXiv receive tens of thousands of new preprints every month, and open indexes such as OpenAlex and Semantic Scholar now describe hundreds of millions of scholarly works [16], [17]. A student starting a literature review, or an engineer checking which method reports the best result on a benchmark, must find the relevant papers across several sources, read them, and record exactly where each claim came from.

Large language models (LLMs) appear to solve this problem: they answer research questions instantly and in fluent prose. However, an LLM answering from its parameters alone is prone to *hallucination*, producing statements, numbers and even references that are not supported by any source [8]. For academic work this is a serious defect, because an answer that cannot be traced to a specific passage cannot be trusted or cited.

Retrieval-augmented generation (RAG) addresses hallucination by retrieving relevant text first and asking the model to answer only from it [2], [11]. Most RAG systems retrieve with dense vector embeddings stored in an approximate-nearest-neighbour index [3], [26], [27]. This works well for paraphrased questions, but it requires an embedding model, a separate vector database, re-embedding whenever the model changes, and it tends to blur exactly the details that matter most in research text: model names, dataset names, units and numerical results. Lexical retrieval based on the probabilistic relevance framework (BM25) remains a strong and robust baseline, particularly outside the domain a dense model was trained on [1], [5].

1.2 Problem Statement

To design and build a research assistant that automatically finds relevant research papers for a natural-language question, answers it using only those papers with every claim cited to the exact paper, section and page, keeps each research topic

up to date as new work appears, and does all of this with lexical retrieval and reranking instead of a vector database.

The problem can be broken into six concrete issues that current practice and tools leave unsolved:

1. **Fragmented discovery.** Relevant papers are spread across arXiv, OpenAlex, Semantic Scholar and publisher sites. Many assistants expect the user to find and upload every source before asking a question.
2. **Unverifiable answers.** LLM answers either carry no citations, cite whole documents, or cite sources that do not exist [8], [9]. A researcher cannot check a claim without the section and page it came from.
3. **Cost and opacity of vector pipelines.** Dense retrieval needs an embedding step for every chunk, a separate vector store, and re-indexing on model changes, and its similarity scores are hard to interpret [26], [27].
4. **Loss of exact terms.** Embeddings of "F1 = 0.95" and "F1 = 0.82" are nearly identical because both mean "an F1 score", so semantic search can confidently return the wrong evidence for precise questions.
5. **Stale knowledge.** A literature review is out of date as soon as it is written; nothing tells the researcher that a new paper on the topic, a new paper citing a key reference, or a revised version of a preprint has appeared.
6. **Isolation from everyday tools.** Researchers increasingly work inside AI assistants and editors, but their research corpus and private PDFs are not available there.

1.3 Motivation

The motivation for IndexR comes from a simple observation: academic text is full of exact identifiers, and the people reading it care about exact evidence. Lexical matching rewards the exact token, so it is well suited to this text, and its weakness (different words for the same idea) can be handled with two well-studied techniques: LLM-based query expansion [13]–[15] and cross-encoder reranking, which judges meaning directly [6], [7]. Combining these with section-aware chunking makes it possible to cite not just a paper but a section and a page, which is exactly what a researcher needs to verify a claim.

A vectorless design is also cheaper and simpler to operate. Updating the index is an ordinary database insert, there is no embedding cost per chunk, and the entire search index lives inside PostgreSQL, a database most teams already run. This makes the approach practical for academic institutions and small teams without GPU servers or large cloud budgets.

1.4 Societal Relevance

- **Research integrity.** Answers whose every sentence is traceable to a section and page discourage the copying of unverified, machine-generated claims into theses, reports and papers.
- **Access for students.** Engineering students in India and elsewhere often lack institutional subscriptions and paid research tools. IndexR builds on open scholarly infrastructure (arXiv, OpenAlex, Semantic Scholar) and only downloads legal open-access copies.
- **Low infrastructure cost.** The system runs on commodity hardware without GPUs or a vector database, so a college or small lab can host it.
- **Keeping knowledge current.** Live tracking helps researchers notice new work early, reducing duplicated effort and outdated literature reviews.
- **Privacy.** Researchers can upload their own drafts and unpublished PDFs, which remain visible only to them.

1.5 Scope of the Project

The scope of Major Project-I covers the complete design, implementation and validation of the system described below. The items outside the scope are recorded so that they can be considered for Major Project-II.

Table 1.1: Scope of the project

In scope	Out of scope
Automatic discovery from arXiv, OpenAlex and Semantic Scholar	Paywalled papers (open access only)
PDF ingestion with section- and page-aware chunking	Table and figure extraction from PDFs
Lexical retrieval, metadata boost, query expansion, reranking	Training a custom or fine-tuned reranker

In scope	Out of scope
Grounded, streamed answers with [n] citations	Multi-user collaboration on one workspace
Workspaces, paper lists, private PDF uploads	Organisation and team accounts
MCP server for AI assistants	A visual citation-graph explorer
Live tracking of topics and papers	Email or push notifications
Administration through the Django admin	Large-scale user study (planned for Project-II)

1.6 Organisation of the Report

Chapter 2 states the objectives of the project in measurable terms. Chapter 3 reviews related work on lexical and dense retrieval, reranking, retrieval-augmented generation, citation and attribution, and scholarly data sources, and identifies the research gap. Chapter 4 presents the proposed methodology: feasibility, system architecture, block and data-flow diagrams, the working principle of each stage and the database and API design. Chapter 5 lists the tools and technologies used. Chapter 6 reports the outcomes, including screenshots, requirement verification, automated testing and performance measurements. Chapter 7 gives the timeline, and Chapter 8 concludes the report and outlines future work.

CHAPTER 2
OBJECTIVES

The primary aim of the project is to build a working, tested research assistant that produces cited answers without a vector database. This aim is divided into the objectives in Table 2.1. Each objective has a measure that can be checked against the delivered system and a target week in the 14-week schedule of Chapter 7.

Table 2.1: Project objectives, success measures and target weeks

No.	Objective	Measure of success	Target
O1	Build a shared, de-duplicated corpus by discovering and ingesting open-access papers from arXiv, OpenAlex and Semantic Scholar.	Papers are stored once per source ID or DOI; every chunk records its heading and page range.	Week 7–8
O2	Implement vectorless retrieval: weighted full-text search with citation and recency boosting, LLM query expansion and cross-encoder reranking.	Median search latency below 50 ms on the development corpus; top-5 passages returned with scores.	Week 7–9
O3	Generate grounded answers whose [n] markers map to the exact paper, section and page, streamed to the user.	Every marker maps to a stored chunk; unsupported questions get an explicit “not enough evidence” reply.	Week 8–9
O4	Decide automatically when the corpus is insufficient and run external discovery (two-gate rule).	Discovery runs only when fewer than 5 chunks match or the best rerank score is below 0.30.	Week 9
O5	Provide a responsive web application for workspaces, chat, papers and settings.	All core flows usable on desktop and mobile, in light and dark themes.	Week 9–10
O6	Support private PDF uploads that only their owner can see.	Automated tests show uploads never appear in other users’ search, retrieval or answers.	Week 10
O7	Expose IndexR to AI assistants through an MCP server with token authentication.	11 tools available; requests without a valid token receive HTTP 401.	Week 10
O8	Track topics and papers for new papers, new citations and new versions.	Daily scheduled check; each update reported once per workspace; re-runs create no duplicates.	Week 10–11
O9	Validate the system thoroughly.	At least 400 automated tests passing; constant query count for every paginated list.	Week 9–12

In addition to these functional objectives, the project follows a set of non-functional goals: the codebase must stay simple and consistent (one service layer shared by the web API, background jobs and MCP server), access control must fail closed, and every review finding must be backed by a regression test.

CHAPTER 3
LITERATURE REVIEW

3.1 Overview

IndexR combines ideas from information retrieval, neural ranking, retrieval-augmented generation and scholarly data infrastructure. This chapter reviews the work in each area that shaped the design, summarises it in Table 3.1, and closes with the research gap the project addresses.

3.2 Lexical Retrieval

Robertson and Zaragoza describe the probabilistic relevance framework and the BM25 ranking function, which scores a document by the frequency of query terms, their rarity across the collection and document length normalisation [1]. Despite its age, BM25 remains the standard lexical baseline. PostgreSQL implements a closely related approach through its full-text search: documents are converted to a `tsvector` of normalised lexemes, which can be weighted by field (for example title above body), queried with a `tsquery`, ranked with `ts_rank`, and indexed with a Generalized Inverted Index (GIN) [23]. IndexR uses this built-in machinery directly, giving section headings weight A and body text weight B.

Thakur et al. introduced the BEIR benchmark of 18 heterogeneous retrieval datasets and showed that BM25 is a robust zero-shot baseline that many dense retrievers fail to beat outside the domain they were trained on, while BM25 followed by a cross-encoder reranker was among the strongest configurations overall [5]. This finding is the main empirical motivation for IndexR's "lexical retrieval + reranking" design.

3.3 Dense Retrieval and Vector Search

Karpukhin et al. proposed Dense Passage Retrieval (DPR), encoding questions and passages with separate BERT encoders and retrieving by inner product, and reported large gains over BM25 on open-domain question answering [3]. Khattab and Zaharia's ColBERT keeps per-token embeddings and scores them with late interaction, improving effectiveness at the cost of a much larger index [4]. For scientific documents, Cohan et al. trained SPECTER on the citation graph to produce document-level embeddings for papers [21].

Dense retrieval requires approximate nearest-neighbour search at scale. Malkov and Yashunin's Hierarchical Navigable Small World (HNSW) graphs [26] and Johnson, Douze and Jégou's GPU-based billion-scale similarity search [27] are the foundations of today's vector databases. These methods are efficient, but they add an embedding model, a separate index to build and keep consistent, and memory-hungry data structures. They also make relevance harder to explain, since similarity in an embedding space has no direct link to the words in the text.

3.4 Neural Reranking

Nogueira and Cho showed that a BERT cross-encoder, which reads the query and a candidate passage together, can rerank BM25 results to state-of-the-art accuracy on MS MARCO [6]. Lin, Nogueira and Yates survey this family of "retrieve then rerank" methods and explain why cross-encoders, though too slow to search a whole corpus, are highly accurate over a short candidate list [7]. IndexR follows this pattern: PostgreSQL produces up to 50 candidates cheaply, and a hosted cross-encoder (Cohere Rerank v3.5 [24]) reorders the top 20 and keeps the best 5.

3.5 Retrieval-Augmented Generation

Lewis et al. coined the term retrieval-augmented generation for models that condition generation on retrieved documents, and showed that it improves factuality on knowledge-intensive tasks [2]. Gao et al. survey the rapidly growing RAG literature, classifying systems as naive, advanced and modular RAG and listing query rewriting, reranking and adaptive retrieval among the most effective improvements [11]. Asai et al.'s Self-RAG trains a model to decide when to retrieve and to critique its own output [10]; IndexR captures a simpler form of this idea with an explicit, rule-based decision about when the corpus is insufficient. Liu et al. found that LLMs use information at the beginning and end of a long context much better than information in the middle [12], which supports passing only a small number of highly ranked passages to the answer step.

3.6 Query Expansion and Rewriting with LLMs

The classic weakness of lexical retrieval is vocabulary mismatch. Recent work uses LLMs to close this gap. Gao et al.'s HyDE generates a hypothetical answer document

and searches with it [13]. Wang, Yang and Wei's Query2doc expands the query with an LLM-generated pseudo-document and reports consistent gains for BM25 [14]. Ma et al. train a small rewriter to reformulate queries for retrieval-augmented LLMs [15]. IndexR applies the same principle in its cheapest form: the LLM rewrites each question into three to five short keyword queries, which are searched together.

3.7 Hallucination, Attribution and Citation

Ji et al. survey hallucination in natural language generation and identify unsupported content as the central risk of LLM output [8]. Nakano et al.'s WebGPT answered questions by browsing and quoting web pages, making the quoted references part of the answer [19]. Bohnet et al. formalised attributed question answering, in which a system must return an answer together with evidence that supports it [20]. Gao et al.'s ALCE benchmark evaluates LLMs on generating text with citations and found that even strong models often cite passages that do not fully support their statements [9]. These works motivate IndexR's strict answer format: passages are numbered, the model may cite only those numbers, unknown numbers are discarded, and each citation is stored with the exact chunk, so the section and page shown to the user are taken from the database rather than from model output.

3.8 Open Scholarly Data Infrastructure

Priem, Piwowar and Orr describe OpenAlex, a fully open index of scholarly works, authors, venues and institutions with a public API that supports filtering by publication date and by citation relationships [16]. Kinney et al. describe the Semantic Scholar open data platform and its Graph API [17], and Lo et al.'s S2ORC provides a large corpus of open-access full text derived from it [18]. arXiv exposes an API for searching and retrieving preprint metadata, including the date of each new version [25]. IndexR uses all three: arXiv and OpenAlex for search, citation lookup and version tracking, and Semantic Scholar as an additional search source.

3.9 Integration with AI Assistants

The Model Context Protocol (MCP) is an open protocol that lets AI assistants call external tools and read external context through a standard client–server interface over stdio or HTTP [22]. Exposing IndexR as an MCP server makes its corpus, cited answers

and tracking available inside assistants such as Claude Desktop and Claude Code and inside code editors, without each client needing custom integration code.

3.10 Summary of Related Work

Table 3.1: Summary of reviewed literature and its relevance to IndexR

Ref.	Work	Key contribution	Relevance / limitation
[1]	Robertson & Zaragoza, 2009	Probabilistic relevance framework and BM25	Basis of lexical ranking; no semantic matching
[5]	Thakur et al., BEIR, 2021	BM25 robust zero-shot; BM25 + reranker strong	Supports vectorless retrieve-then-rerank
[3]	Karpukhin et al., DPR, 2020	Dual-encoder dense retrieval	Needs embeddings and vector index
[4]	Khattab & Zaharia, ColBERT, 2020	Late-interaction token embeddings	Large index; heavy infrastructure
[26], [27]	HNSW; FAISS GPU search	Approximate nearest-neighbour search	Extra memory and services to operate
[6], [7]	Nogueira & Cho; Lin et al.	Cross-encoder reranking of candidates	Adopted for the top-20 candidates
[2], [11]	Lewis et al.; Gao et al. survey	Retrieval-augmented generation	Usually built on dense retrieval
[10]	Asai et al., Self-RAG, 2024	Model decides when to retrieve	Needs model training; IndexR uses a rule
[12]	Liu et al., 2024	LLMs underuse mid-context passages	Justifies passing only top-5 passages
[13]–[15]	HyDE; Query2doc; query rewriting	LLM query expansion	Adopted as keyword query expansion
[8], [9]	Ji et al.; Gao et al., ALCE	Hallucination; citation quality	Citations often unsupported or coarse
[19], [20]	WebGPT; attributed QA	Answers paired with evidence	Evidence at web-page level
[16]–[18], [25]	OpenAlex; Semantic Scholar; S2ORC; arXiv	Open scholarly metadata and APIs	Used for discovery, citations, versions
[22]	Model Context Protocol	Standard tool interface for assistants	Used to expose IndexR's tools

3.11 Research Gap Analysis

The reviewed literature shows strong individual techniques but leaves several gaps when the goal is a practical, trustworthy research assistant:

- **G1 – Dependence on vector infrastructure.** Most RAG systems assume dense retrieval [2], [3], [11], even though lexical retrieval with reranking is competitive and more robust out of domain [5]. Few systems demonstrate a complete, production-style RAG pipeline without a vector database.
- **G2 – Coarse or unreliable citations.** Systems that cite usually cite whole documents or web pages [19], [20], and LLM-generated citations are often unsupported [9]. Citation at section and page level, with the location taken from stored data rather than from model output, is rarely provided.
- **G3 – Static corpora.** RAG systems answer from a fixed, user-supplied collection; they neither discover new papers automatically nor decide when their own corpus is insufficient (Self-RAG [10] requires a specially trained model).
- **G4 – No awareness of change.** None of the reviewed systems notify the user of new papers on a topic, new citations of a key paper, or new versions of a preprint, although open APIs make this information available [16], [25].
- **G5 – Isolation.** Research corpora are rarely available inside the AI assistants researchers already use, despite a standard protocol for this now existing [22].

IndexR addresses all five gaps in one system: G1 through PostgreSQL full-text search, LLM query expansion and cross-encoder reranking; G2 through section-aware chunking and server-side citation mapping; G3 through the two-gate discovery rule; G4 through live tracking; and G5 through its MCP server.

CHAPTER 4
PROPOSED METHODOLOGY

4.1 Approach

IndexR is built as a pipeline of independent services behind a single service layer. The same services are called by the REST API used by the web application, by background Celery tasks, and by the MCP server used by AI assistants, so every entry point applies the same retrieval logic and the same access rules. The methodology followed the milestones of the college template: problem definition and literature survey, a feasibility study, system design with block and flow diagrams, implementation in incremental phases, and design validation through automated and live testing.

4.2 Feasibility Study

4.2.1 Technical Feasibility

All required capabilities are available as mature open-source software or public APIs. PostgreSQL provides weighted full-text search and GIN indexes [23]; Django and Django REST Framework provide the web API; Celery and Redis provide background processing and scheduling; `pymupdf4llm` converts PDFs to Markdown with page boundaries; arXiv, OpenAlex and Semantic Scholar provide open metadata and citation data [16], [17], [25]; hosted LLM and reranking APIs provide language understanding without local GPUs. The team was already familiar with Python, Django and React.

4.2.2 Economic Feasibility

The project needs no special hardware. Development was done on a single laptop, and all software used is free and open source. The external scholarly APIs are free; the LLM and reranker are pay-per-use services with free development tiers, and the pipeline limits their use (one query-expansion call, one rerank call over at most 20 passages and one answer call per question). Removing the vector database removes the cost of an embedding model and of a separate vector store.

4.2.3 Operational Feasibility

Users interact through a familiar chat-style web interface and need no training in search syntax. Administrators use the standard Django admin. The system runs as four processes (web server, Celery worker, Celery beat scheduler and the optional MCP

server) on top of PostgreSQL and Redis, which are widely supported by hosting providers.

4.2.4 Schedule Feasibility

The work was split into nine small phases, each ending in a reviewed merge (Chapter 7), so that a usable product existed early and later features (uploads, MCP, tracking) were added on top without destabilising the core pipeline.

4.3 System Architecture

Fig. 4.1 shows the overall architecture. Clients (the React web application, AI assistants using MCP, and staff using the Django admin) reach the backend over HTTPS. The web application authenticates with short-lived JSON Web Tokens; AI assistants authenticate with personal access tokens. Both the REST API and the MCP server delegate to the same service layer, which reads and writes PostgreSQL, queues long-running work through Redis to Celery workers, and calls the external paper sources, the LLM provider and the reranker.

The main components are:

- **Web application** – a single-page React application (Vite, TanStack Query, shadcn/ui) for workspaces, chat, papers, tracking and settings.
- **REST API** – Django REST Framework view sets organised by app: `iam` (accounts), `papers` (search and uploads), `workspaces` (workspaces, papers, messages, topics, updates) and `integrations` (access tokens).
- **Streaming endpoint** – answers are streamed to the browser as Server-Sent Events (SSE) so the user sees progress and text as soon as they are available.
- **Service layer** – `AskService`, `RetrievalService`, `DiscoveryService`, `AnswerService`, `RerankService`, `QueryExpansionService`, `PaperStoreService`, `PaperUploadService` and `TrackingService`.
- **Celery worker and beat** – ingest PDFs in the background, run the daily tracking check, and requeue stalled ingestions every hour.
- **MCP server** – exposes eleven tools over stdio for local use and over streamable HTTP for remote clients.
- **PostgreSQL** – stores all data and the full-text index; **Redis** – the task broker.

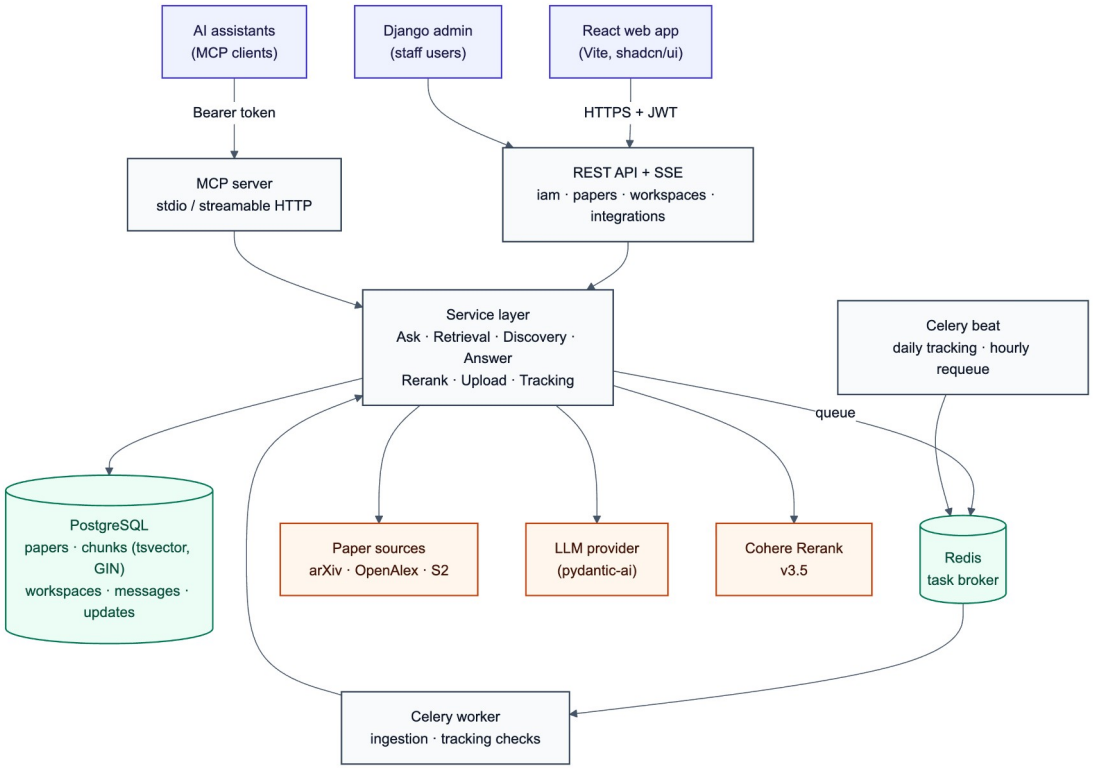


Figure 4.1: System architecture of IndexR

4.4 Block Diagram

Fig. 4.2 presents the functional blocks of the question-answering pipeline in three stages. The central loop is retrieve → check → (discover → ingest → retrieve again) → answer. Private uploads enter the same ingestion stage, and live tracking reuses the discovery stage (Sections 4.8 and 4.9).

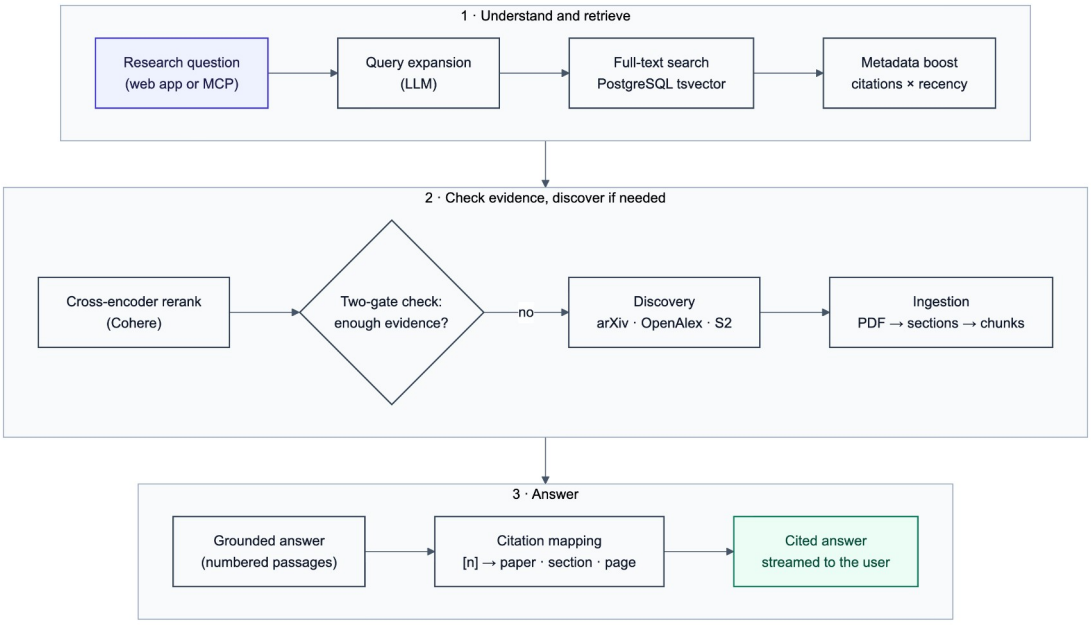


Figure 4.2: Block diagram of the IndexR pipeline

4.5 Data Flow Diagrams

The context-level data flow diagram (Fig. 4.3) treats IndexR as a single process exchanging data with three external entities (the researcher, AI assistants and the administrator) and two groups of external services. The level-1 diagram (Fig. 4.4) decomposes the system into four processes and four data stores.

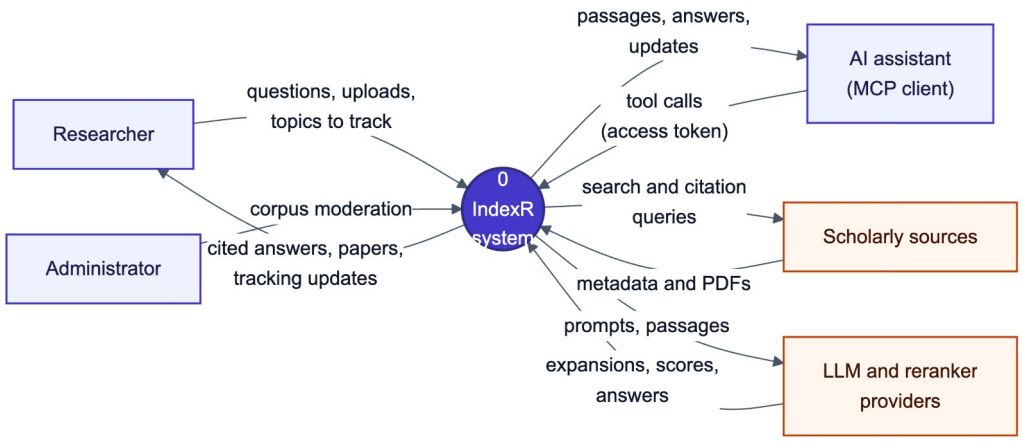


Figure 4.3: Level-0 (context) data flow diagram

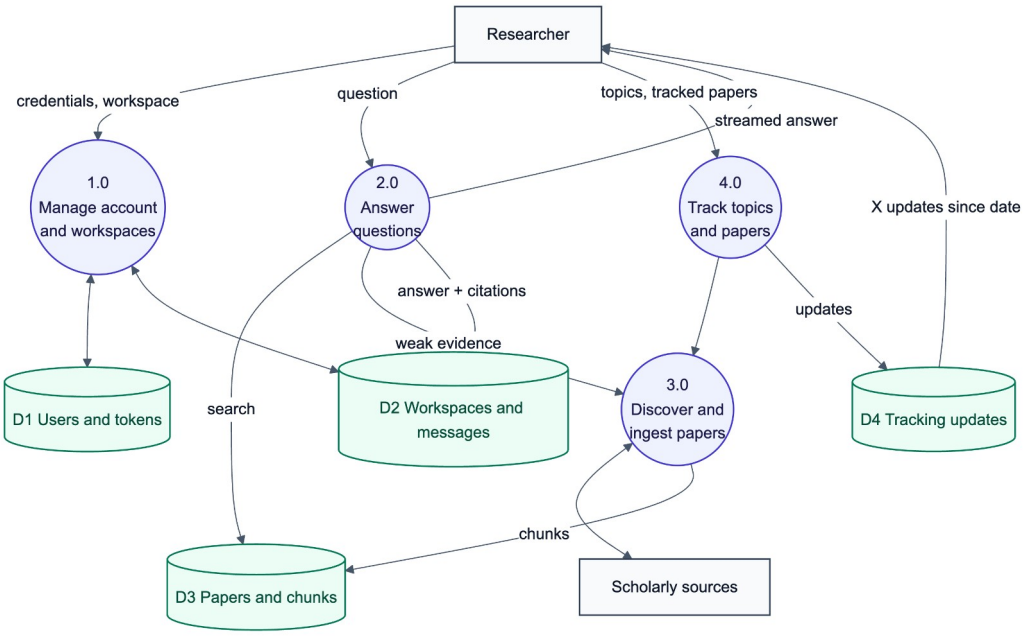


Figure 4.4: Level-1 data flow diagram

4.6 Use Cases

Fig. 4.5 summarises what each actor can do. Researchers manage workspaces, ask questions, inspect citations, manage papers, track topics and papers, and create access tokens for their assistants. AI assistants use the MCP tools on a researcher's behalf. Administrators moderate the shared corpus.

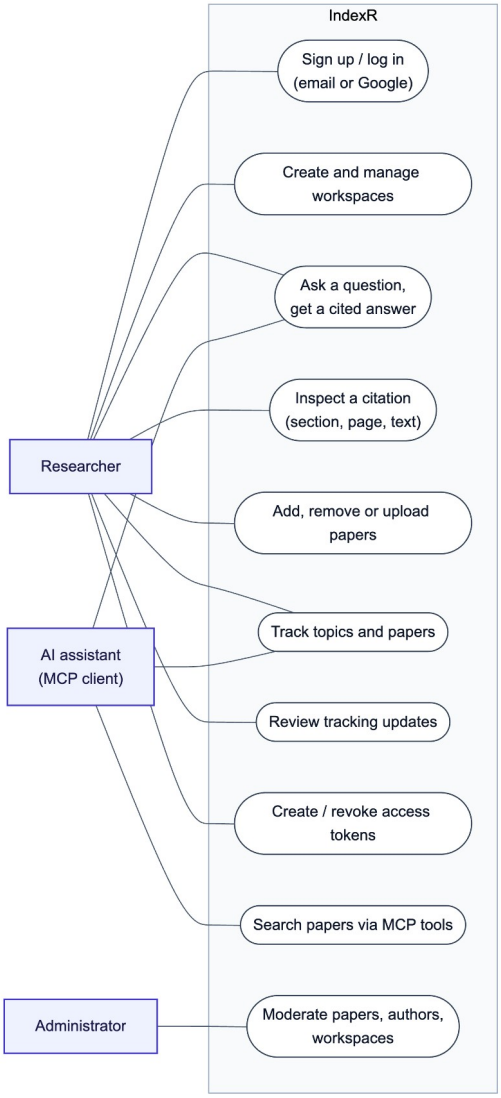


Figure 4.5: Use-case diagram

4.7 Working Principle

The complete flow of a question is shown in Fig. 4.6. The following subsections describe each stage.

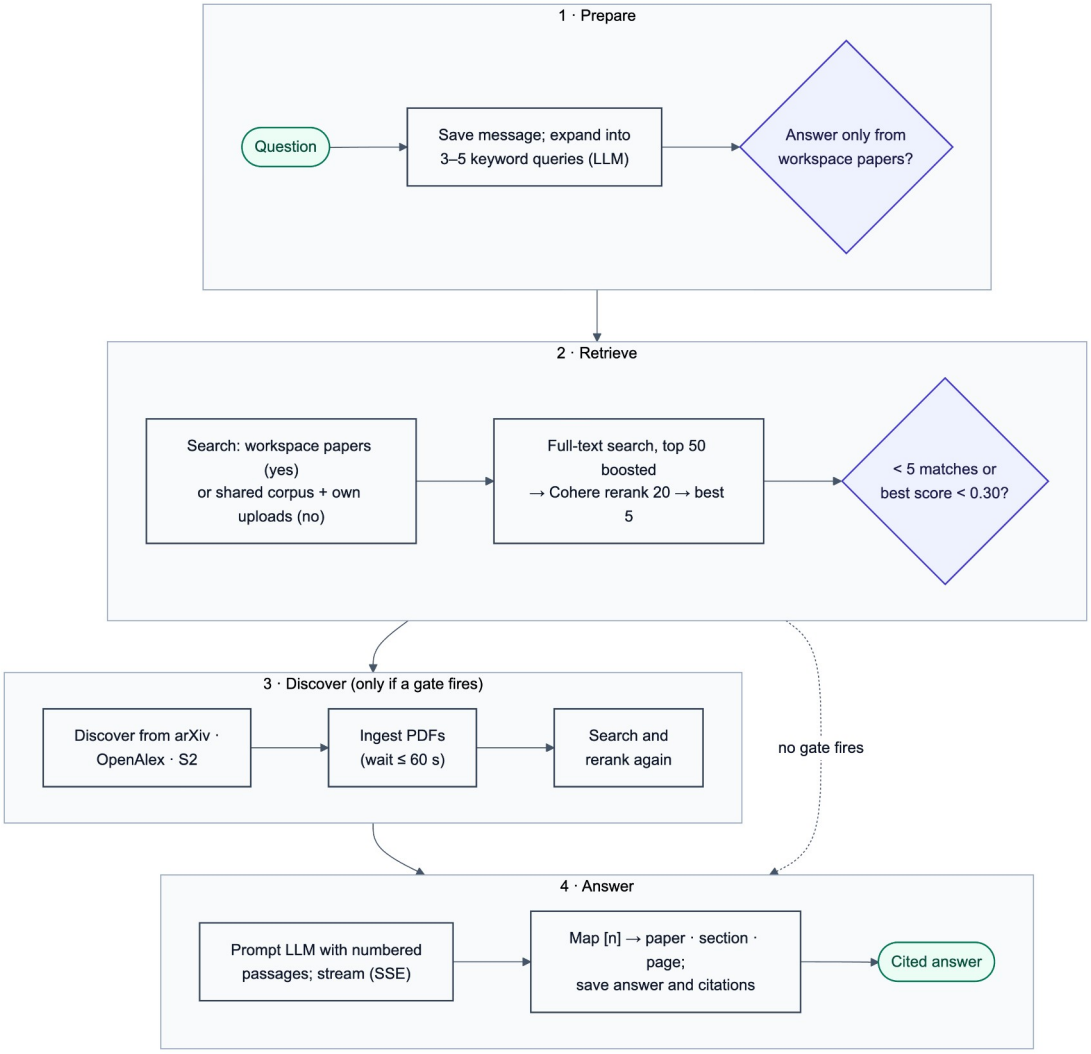


Figure 4.6: Flowchart of answering a question

4.7.1 Paper Discovery

When discovery is needed, `DiscoveryService` sends every expanded query to arXiv, OpenAlex and Semantic Scholar in parallel (10 results per source per query). Results are de-duplicated across sources by DOI and by a normalised title. To avoid filling the corpus with off-topic papers, each candidate's title and abstract are scored against the question with the cross-encoder, and only papers scoring at least 0.25 are kept, up to 10 per question. Kept papers are stored with their authors by `PaperStoreService`, which reuses an existing row when the same paper is found again (by source ID or DOI) and fills in any missing fields. A paper with an open-access PDF is queued for ingestion.

4.7.2 Ingestion and Section-Aware Chunking

Ingestion runs in a Celery worker (Fig. 4.7). The PDF is downloaded (at most 50 MB, and only if it starts with the `%PDF` signature) and converted to Markdown page by page

with `pymupdf4llm`, which keeps headings. The text is split into blocks and grouped under its section headings; reference lists and acknowledgements are dropped. Sections are packed into chunks of at most 1,000 characters, splitting at paragraphs, then lines, sentences and words, and every chunk keeps its heading and the page range it spans. A database trigger maintains each chunk's `tsvector` with the heading at weight A and the text at weight B, so a paper is searchable as soon as its chunks are stored.

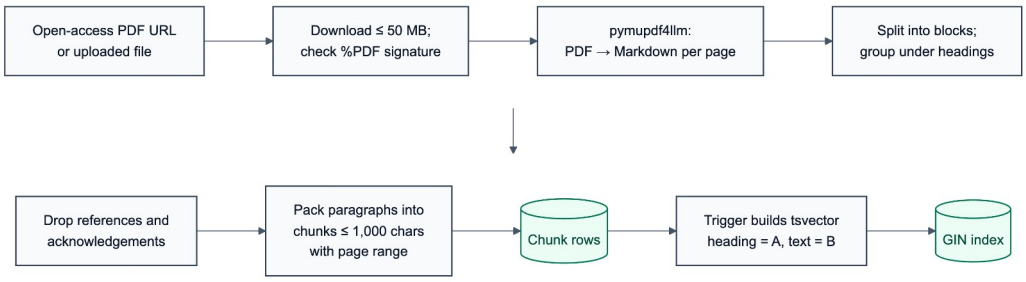


Figure 4.7: Ingestion and chunking pipeline

4.7.3 Lexical Ranking with Metadata Boost

Expanded queries are combined with OR into a single PostgreSQL web-search query, and matching chunks are ranked by `ts_rank`. Because research quality and recency matter, the rank is multiplied by two boosts computed from the paper's metadata:

$$\text{score} = \text{rank} \times \text{B_cit} \times \text{B_rec} \tag{4.1}$$

$$\text{B_cit} = 1 + 0.1 \times \log_{10}(c + 1) \tag{4.2}$$

$$\text{B_rec} = 1 + 0.2 \times \max(0,\ 1 - a\ /\ 5) \tag{4.3}$$

where c is the paper's citation count and a its age in years. A paper with 1,000 citations therefore receives a 30% boost, and a paper published this year receives a 20% boost that falls linearly to zero at five years; undated papers are not boosted. The top 50 chunks by score are the candidates. Optional filters restrict candidates by publication year, venue and minimum citation count.

4.7.4 Query Expansion

Before searching, the LLM rewrites the question into three to five short keyword queries that include synonyms and the specific terms a paper would use (for example, a question about "making transformers handle long inputs" is expanded to queries about sparse attention, linear attention and long-context models). This addresses the

vocabulary mismatch of lexical search in the spirit of [13]–[15] while keeping retrieval itself fully lexical.

4.7.5 Cross-Encoder Reranking

The top 20 candidates are sent with the original question to Cohere Rerank v3.5, a cross-encoder that reads the question and each passage together and returns a relevance score between 0 and 1 [6], [24]. The five best passages are kept for the answer. Reranking corrects the main weakness of lexical ranking: a passage that merely mentions the query terms ranks below one that actually answers the question.

4.7.6 Two-Gate Discovery Rule

Lexical scores are unbounded and their scale changes with query and document length, so a fixed threshold on them would not generalise. IndexR instead decides whether its corpus is sufficient with the two gates in Table 4.1. If either gate fires, and the workspace is not restricted to its own papers, discovery runs, newly found PDFs are ingested (waiting up to 60 seconds), and retrieval is repeated.

Table 4.1: The two-gate rule for triggering discovery

Gate	Check	Meaning
1. Count	Fewer than 5 candidate chunks match the expanded queries	The corpus has no real coverage of the topic
2. Confidence	The best cross-encoder score is below 0.30	Words matched, but no passage answers the question

4.7.7 Grounded Answer Generation and Citation Mapping

The five passages are numbered [1]–[5] and given to the LLM together with the question and strict instructions: answer only from the passages, cite each claim with its passage number, and say so plainly when the passages are not enough. The model returns structured output (the answer text and a flag for insufficient context), validated with `pydantic-ai`. After generation, every `[n]` marker in the text is mapped to the chunk that was numbered *n*; markers that do not correspond to a passage are dropped. The answer is stored as a message together with `MessageCitation` rows linking each number to its chunk, and the cited papers are added to the workspace. Because the section and page shown to the user come from the stored chunk, the model cannot invent a location.

4.7.8 Streaming the Answer

The streaming endpoint sends the stages of the pipeline to the browser as Server-Sent Events: the saved question, status updates (“Understanding your question”, “Looking through papers”, “Finding new papers”, “Reading new papers”, “Writing the answer”), the growing answer text, and finally the saved answer with its citations (Fig. 4.8). If anything fails, a friendly error event is sent instead.

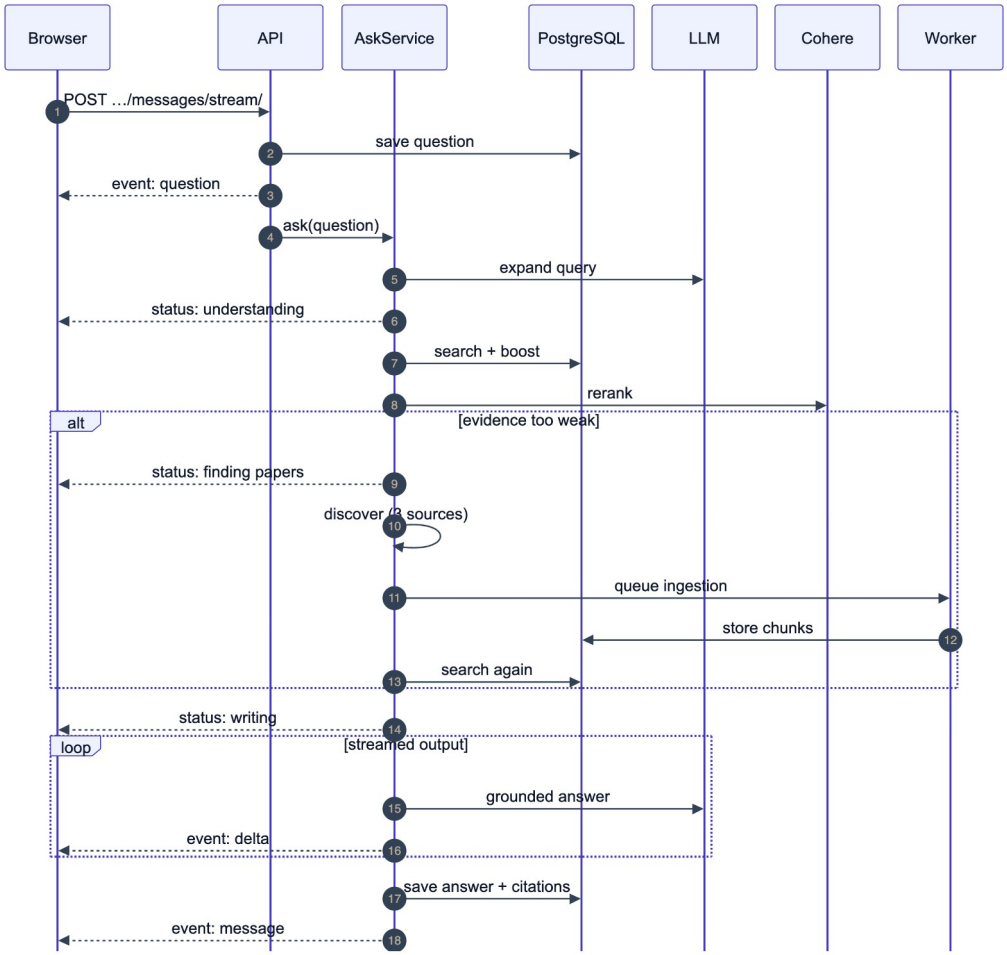


Figure 4.8: Sequence diagram of a streamed answer

4.8 Private Uploads and Visibility

A user can upload their own PDF to a workspace. The file is validated, extracted and chunked synchronously with the same extractor, and stored as a paper whose `owner` is the uploader; the PDF itself is not kept. Every query that reads papers or chunks goes through `visible_to(user)`, which returns the shared corpus plus that user’s own uploads, so private papers never appear in another user’s search results, retrieval, answers, workspace additions or MCP tools. Database constraints guarantee that

uploads are always private and that the same file uploaded twice by one user is stored once (by SHA-256 content hash).

4.9 Live Tracking

A workspace can track up to ten topics, and any shared paper in a workspace can be tracked. Celery beat runs a daily check (Fig. 4.9) that fans out into one task per topic and per tracked paper:

- **New papers on a topic** – the discovery pipeline runs with a date filter so that only papers published since the topic was last checked are returned (30 days back on the first check). Papers already in the workspace are skipped.
- **New citations of a paper** – the paper's OpenAlex work is found (directly, or by DOI, or by its arXiv DOI), and OpenAlex's `cites:` filter lists the ten newest citing papers since the last check. Each citing paper is stored in the corpus and linked by a `Citation` edge.
- **New versions of a paper** – arXiv is asked, in batches, for the latest revision date of every tracked arXiv paper; a date newer than the stored one is a new version.

Each find is stored as a `TrackingUpdate` for the workspace. Unique constraints ensure that a new paper or citation is reported once per workspace, so repeated checks never create duplicates. The workspace stores when its updates were last opened (`updates_seen_at`); the number of unseen updates is derived from it and shown as an "N updates since date" banner and as a badge on the home page.

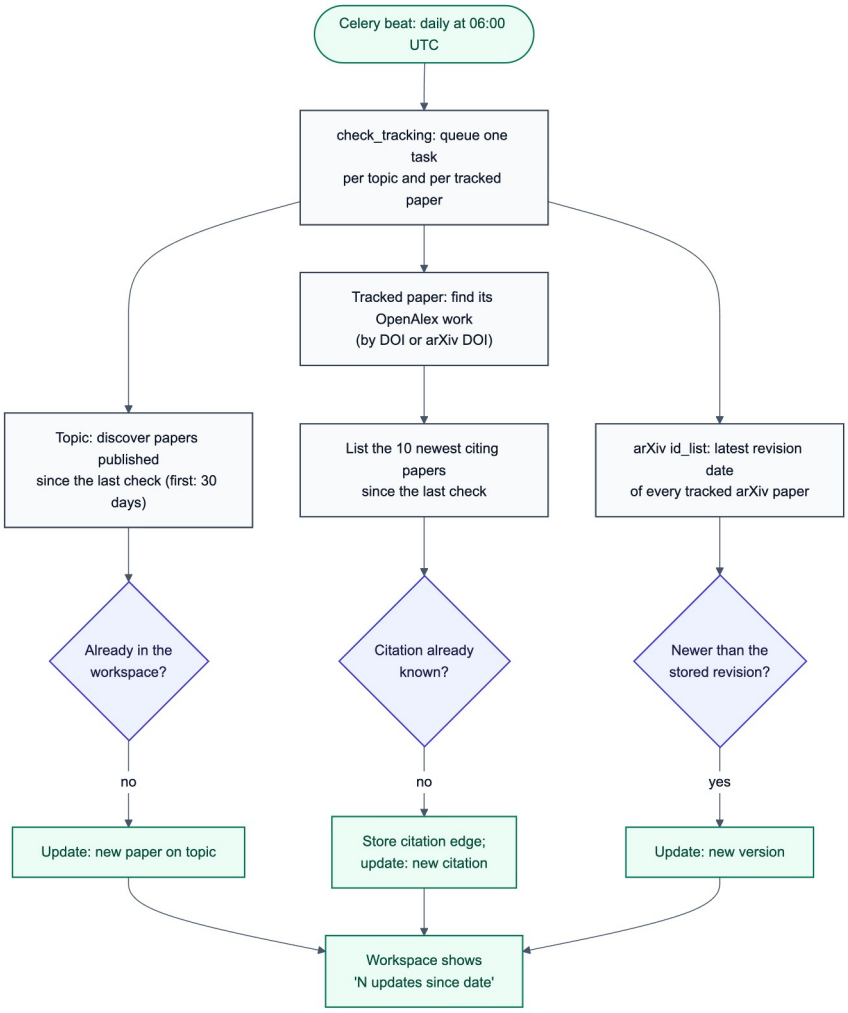


Figure 4.9: Flowchart of the daily tracking check

4.10 MCP Integration

The MCP server is built with the official Python `mcp` SDK and runs either locally over stdio (acting as one user, given by an access token in the environment) or remotely over streamable HTTP. Remote clients send a personal access token as a bearer token; the server stores only the token's SHA-256 hash, verifies it on every request and acts as that token's user (Fig. 4.10). Users create, name and revoke tokens in the web application, which also shows ready-made configuration for common clients. The tools are listed in Table 4.2.

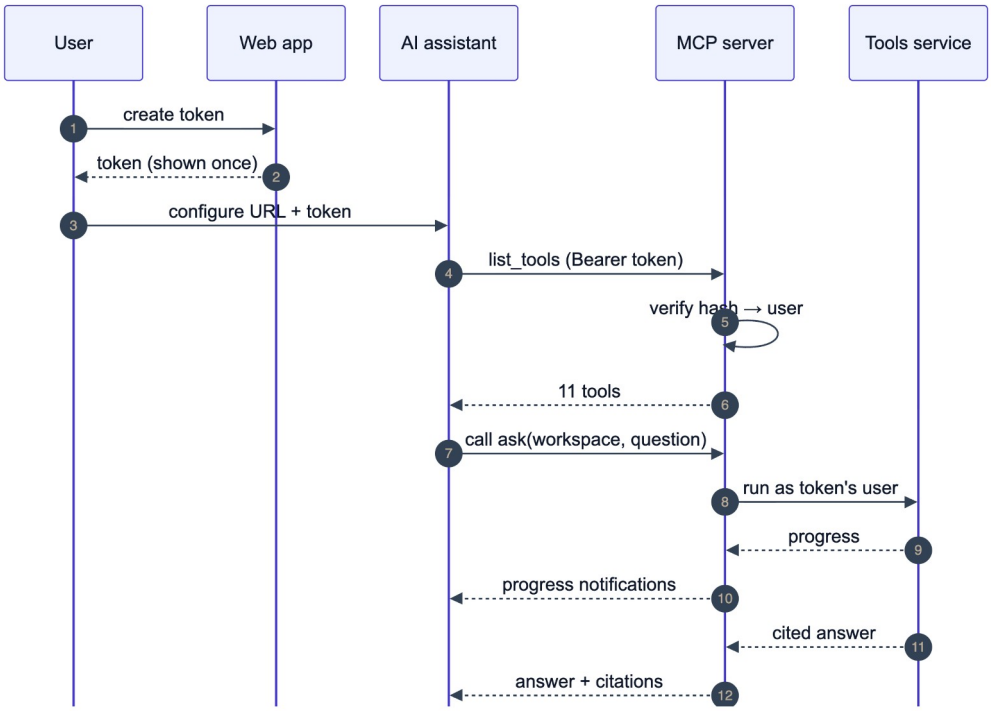


Figure 4.10: Connecting an AI assistant through MCP

Table 4.2: Tools exposed by the IndexR MCP server

Tool	Purpose
search_papers	Search passages in the shared corpus and the user's uploads, with year, venue and citation filters
ask	Ask a question in a workspace; returns the cited answer and reports progress while working
list_workspaces	List the user's workspaces, most recently active first
create_workspace	Create a workspace for a research topic
list_workspace_papers	List the papers in a workspace
get_paper	Get a paper's details: title, authors, venue, year, citations and links
add_paper_to_workspace	Add a paper the user can see to a workspace
remove_paper_from_workspace	Remove a paper from a workspace
track_topic	Start tracking a topic in a workspace
untrack_topic	Stop tracking a topic
list_updates	List what tracking found, optionally only unseen updates

4.11 Database Design

The entity–relationship diagram is shown in two parts: Fig. 4.11 covers users, access tokens and the shared corpus of papers, authors, chunks and citations, and Fig. 4.12 covers workspaces with their papers, chat messages, citations and tracking. The schema follows a few deliberate rules: values that can be derived are not stored (for example, ingestion status is derived from the `ingested_at` and `ingestion_failed_at` timestamps, and an update's kind from which of its references is set); events are recorded as timestamps rather than status columns; every rule the database can express is a check or unique constraint; and many-to-many relationships with attributes use explicit through models (Table 4.3).

Table 4.3: Main database entities

Entity	App	Purpose
User	iam	Email-based account; staff users can use the admin
Paper	papers	Source, external ID, DOI, metadata, revision date, ingestion timestamps; private when it has an owner
Author, PaperAuthor	papers	Authors (with optional ORCID) and their byline order
Chunk	papers	Section-aligned text with heading, page range and weighted search vector
Citation	papers	Paper → cited-paper edge, filled by citation tracking
Workspace	workspaces	A user's topic space; "answer only from these papers" switch; when updates were last seen
WorkspacePaper	workspaces	Papers kept in a workspace; when tracking of the paper started
Message, MessageCitation	workspaces	Chat messages and the chunk each [n] refers to
TrackedTopic	workspaces	Topic query, up to 10 per workspace, with time of last check
TrackingUpdate	workspaces	New paper, new citation or new version found for a workspace
ApiToken	integrations	Named personal access token; only hash and prefix stored

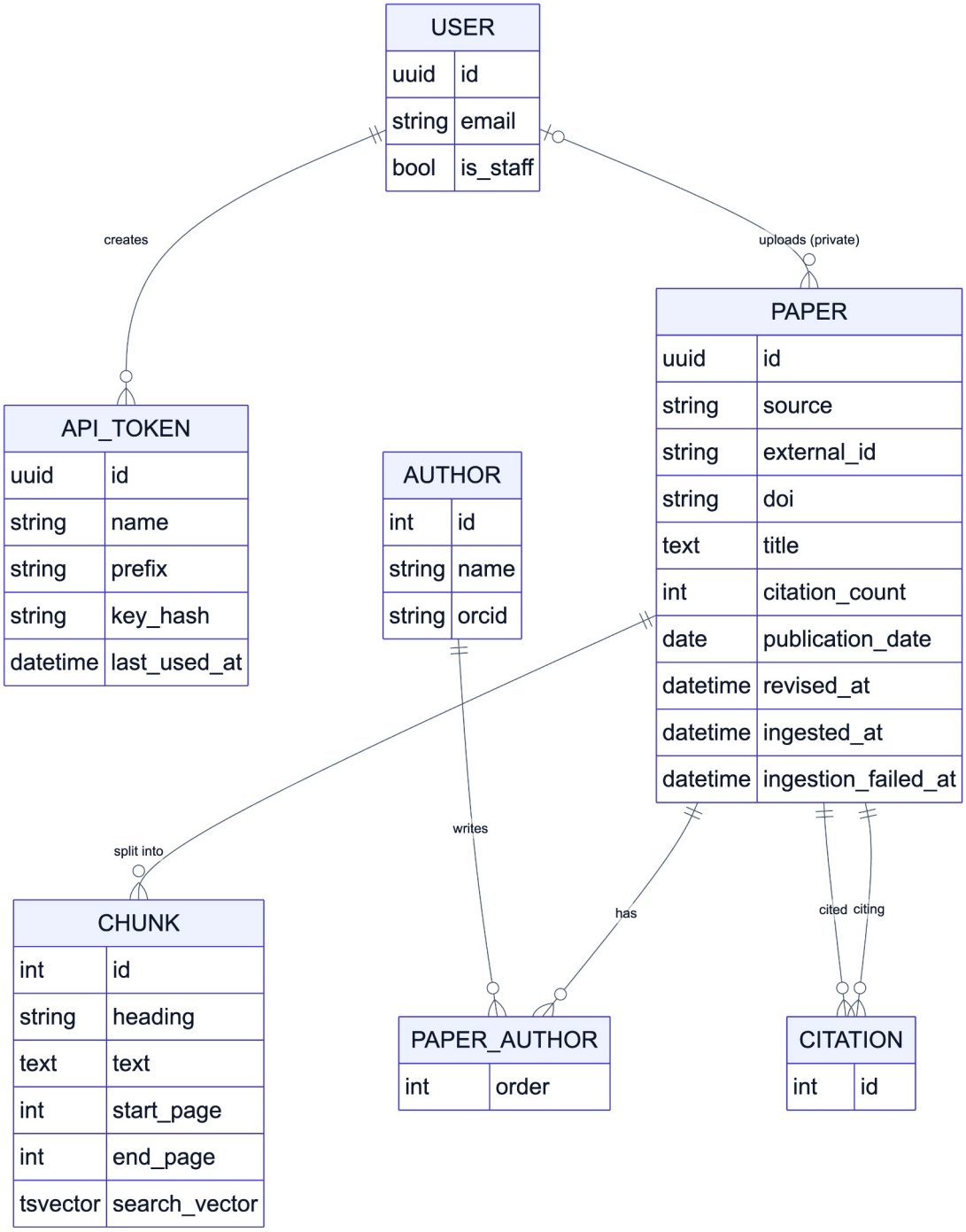


Figure 4.11: Entity–relationship diagram: users, papers and the corpus

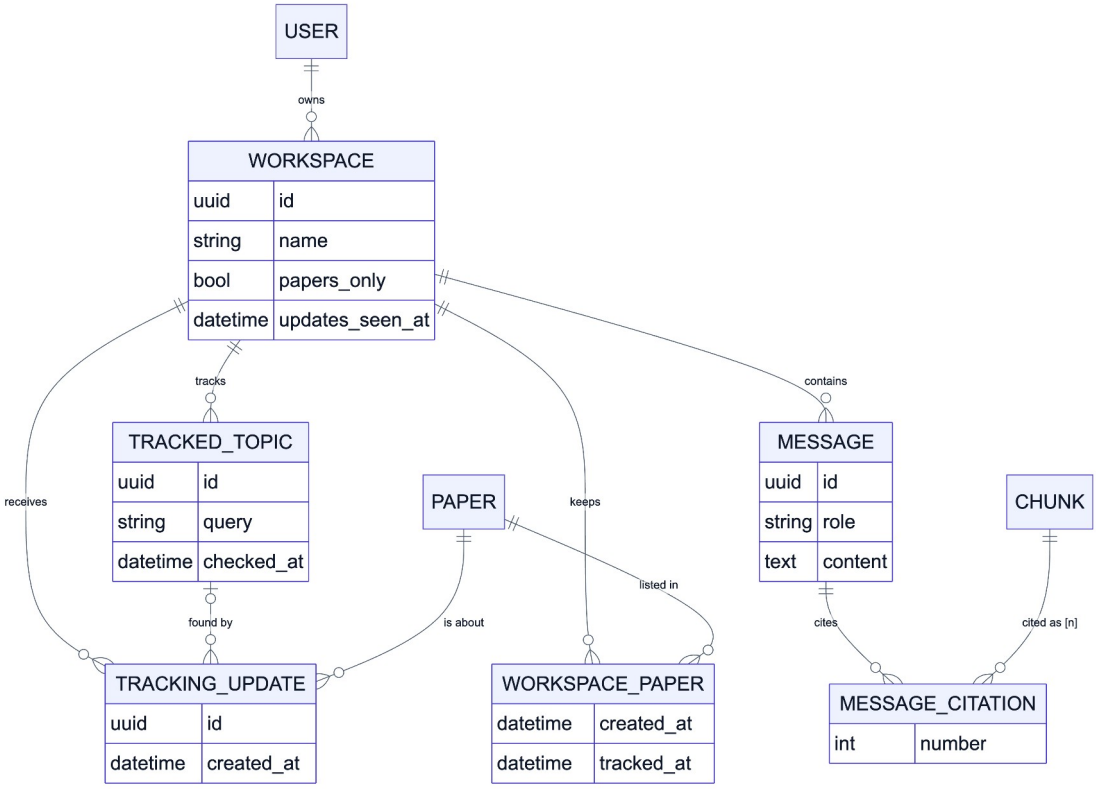


Figure 4.12: Entity–relationship diagram: workspaces, chat and tracking

4.12 API Design

The REST API is grouped by app, uses cursor pagination with a fixed ordering for every list, and validates input in serializers so that errors are returned as HTTP 400 responses with a field name. The main endpoints are listed in Table 4.4.

Table 4.4: Main REST API endpoints

Endpoint	Methods	Purpose
/api/iam/auth/…	POST	Sign-up, login, token refresh, Google sign-in
/api/papers/	POST	Upload a private PDF
/api/papers/chunks/search/?q=	GET	Full-text passage search with filters
/api/workspaces/	GET, POST, PATCH, DELETE	Workspace CRUD
/api/workspaces/{id}/papers/	GET, POST, PATCH, DELETE	Workspace papers; track or untrack a paper
/api/workspaces/{id}/messages/	GET, POST	Chat history; ask (JSON)
/api/workspaces/{id}/messages/stream/	POST	Ask with a streamed (SSE) answer
/api/workspaces/{id}/mentioned-papers/	GET	Papers cited by the workspace's answers

Endpoint	Methods	Purpose
/api/workspaces/{id}/topics/	GET, POST, DELETE	Tracked topics
/api/workspaces/{id}/updates/	GET; POST seen/	Tracking updates; mark them seen
/api/integrations/tokens/	GET, POST, DELETE	Personal access tokens

4.13 Security and Access Control

- **Authentication** – JWT access tokens with refresh-token rotation and blacklisting for the web application; email verification; Google sign-in; hashed personal access tokens for MCP clients.
- **Authorisation** – a shared permission base class requires login, and an ownership check guards every object. Querysets are scoped to the caller before any lookup, so another user's workspace returns HTTP 404 rather than revealing its existence.
- **Fail-closed relations** – fields that accept a related ID (for example the paper to add to a workspace) default to an empty queryset and are widened only to what the caller may reach.
- **Read-only fields** – every serializer declares which fields are read-only, so clients cannot set owners, timestamps or computed values.
- **Safe downloads and uploads** – size limits and PDF signature checks on every PDF; uploaded files are not stored.
- **MCP transport security** – DNS-rebinding protection with allowed hosts and origins.

CHAPTER 5
TOOLS AND TECHNOLOGIES

5.1 Software Stack

Table 5.1 lists the main technologies used, with the versions used during development.

Table 5.1: Software stack

Layer	Technology (version)	Purpose
Language (backend)	Python 3.12	Backend, workers, MCP server
Web framework	Django 6.1, Django REST Framework 3.18	REST API, ORM, admin
Authentication	django-allauth 65, dj-rest-auth 7.2, Simple JWT 5.5	Accounts, email verification, Google sign-in, JWT
Database	PostgreSQL 18	Relational data and full-text search (tsvector, GIN)
Task queue	Celery 5.6 with Redis 8	PDF ingestion, scheduled tracking checks
PDF processing	pymupdf4llm 1.28	PDF to Markdown with headings and pages
LLM interface	pydantic-ai 2.64	Provider-independent LLM calls with validated, streamed output
Reranker	Cohere Python SDK 7.3 (Rerank v3.5)	Cross-encoder scoring of passages and abstracts
Assistant integration	mcp 2.3 (Python SDK)	MCP server over stdio and streamable HTTP
Language (frontend)	TypeScript 6	Type-safe web application
UI framework	React 19, React Router 8	Single-page application
Build tool	Vite 8	Development server and production build
Data fetching	TanStack Query 5, Axios	Caching, pagination, mutations
Styling and components	Tailwind CSS 4, shadcn/ui, Base UI	Accessible, themed components with light and dark modes
Diagrams and testing	Mermaid, Playwright	Report diagrams; automated browser checks of the UI

5.2 External Services and APIs

Table 5.2: External services used by IndexR

Service	Use in IndexR
arXiv API	Search preprints (with a submission-date filter for tracking) and read the latest revision date of tracked papers

Service	Use in IndexR
OpenAlex API	Search works (with a publication-date filter), resolve works by DOI, list citing works
Semantic Scholar Graph API	Additional paper search source (optional API key for higher rate limits)
LLM provider	Query expansion and grounded answers; configurable as provider:model (e.g. Google Gemini, Anthropic Claude, OpenAI GPT)
Cohere Rerank v3.5	Reranking candidate passages and filtering discovered abstracts
Google Identity Services	Optional Google sign-in and account connection

5.3 Development Environment

Table 5.3: Development and test environment

Item	Specification
Hardware	Apple M2 processor, 16 GB RAM (no GPU required)
Operating system	macOS 15.5
Runtimes	Python 3.12.10 (Pipenv virtual environment), Node.js 24
Services	PostgreSQL 18.3, Redis 8
Version control	Git and GitHub, one branch and reviewed pull request per phase
Testing	Django test runner (unit, integration, API and query-count tests); Playwright with Chrome for UI checks
Editors	PyCharm / VS Code

5.4 Deployment Requirements

A modest virtual server is sufficient for a department-level deployment: 2 virtual CPUs, 4 GB of RAM and 20 GB of storage comfortably run the Django application server, one Celery worker with four threads, Celery beat, the MCP server, PostgreSQL and Redis. No GPU is needed because language understanding and reranking are provided by hosted APIs. Configuration (database credentials, API keys, the LLM to use, the tracking hour and the public MCP URL) is supplied through environment variables.

CHAPTER 6
EXPECTED OUTCOMES AND VALIDATION

The project has been implemented in full, so this chapter reports the outcomes achieved rather than only the outcomes expected: the deliverables, the user interface, the verification of each requirement, the automated test results, performance measurements and live runs against the real external services, followed by the metrics proposed for the evaluation in Major Project-II.

6.1 Deliverables

Table 6.1: Project deliverables

Deliverable	Description
Backend service	Django REST API with apps iam, papers, workspaces and integrations (about 9,200 lines of Python including tests)
Web application	React single-page application (about 5,500 lines of TypeScript, excluding generated UI primitives)
Background processing	Celery tasks for ingestion, tracking and requeueing, with a beat schedule
MCP server	Eleven tools, stdio and streamable HTTP transports, token authentication
Test suite	416 automated tests
Documentation	README with setup steps, project overview, diagrams and this report

6.2 User Interface

Fig. 6.1 to Fig. 6.13 show the main screens of the web application, captured from the running system with a demonstration account.

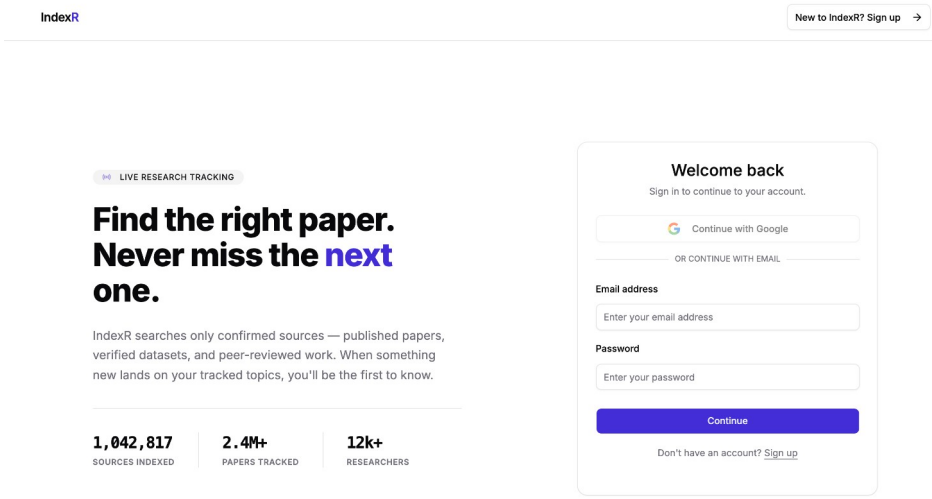


Figure 6.1: Login page

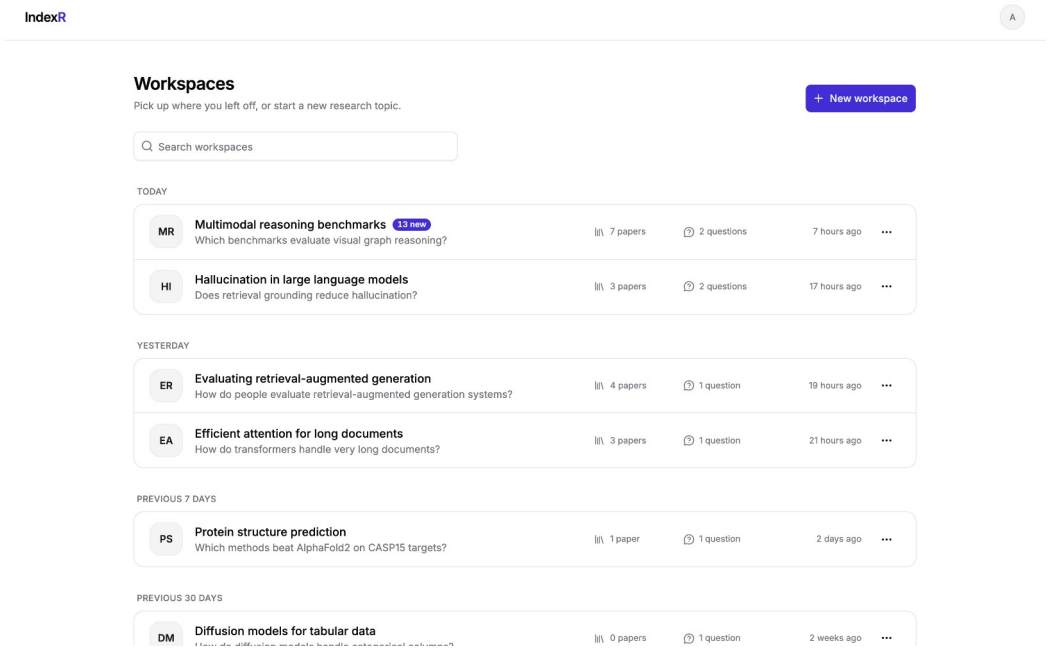


Figure 6.2: Home page listing workspaces, with the unseen-updates badge

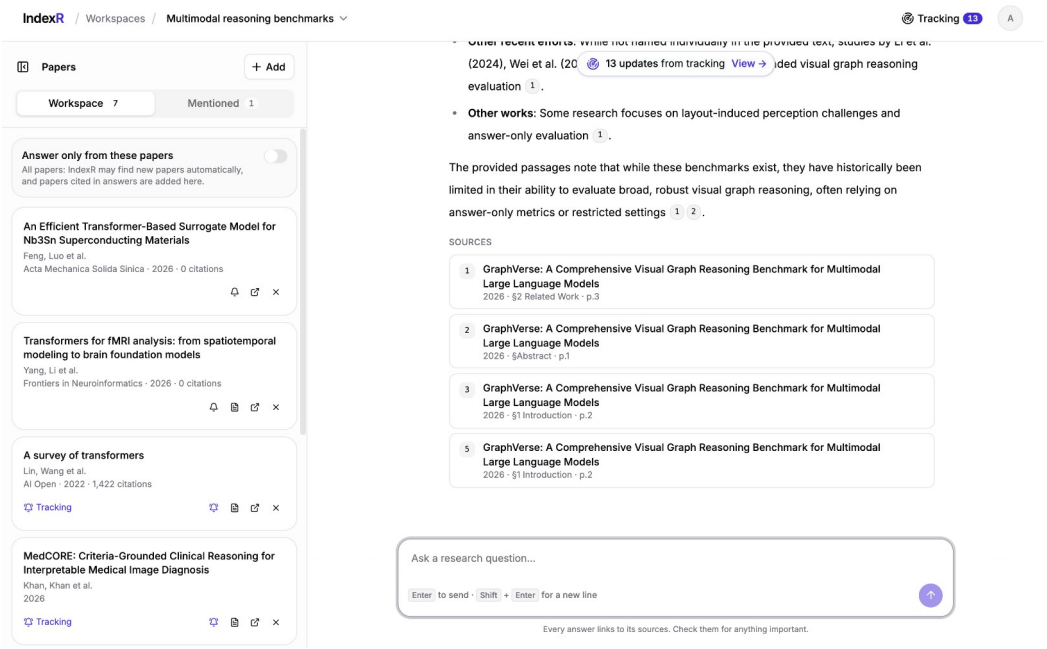


Figure 6.3: Workspace with the papers panel and a cited answer with its sources

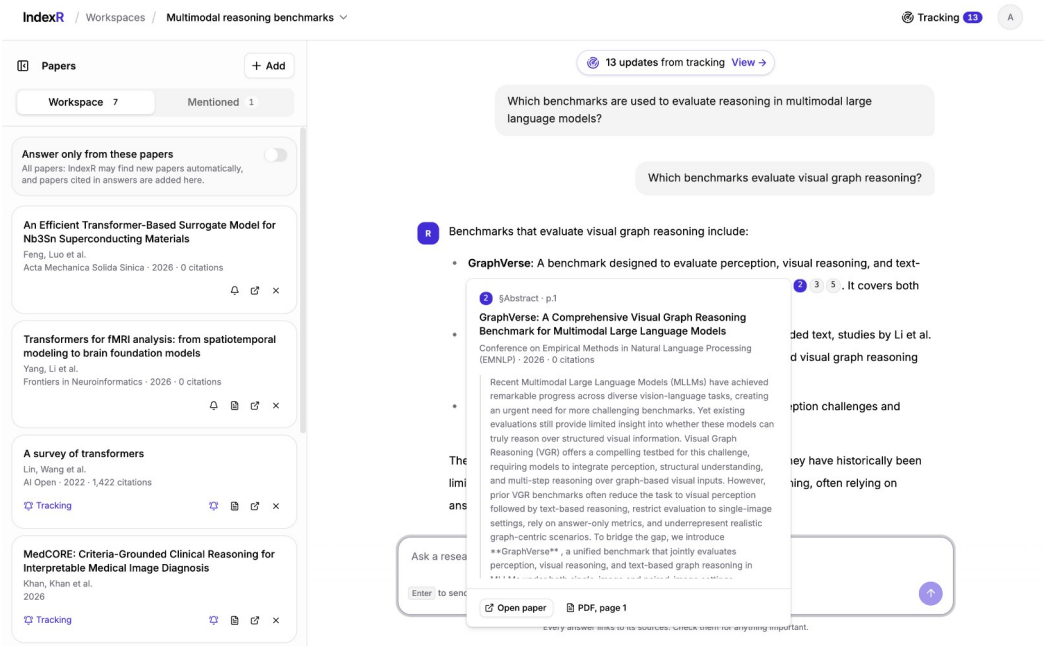


Figure 6.4: Citation preview showing the paper, section, page and passage

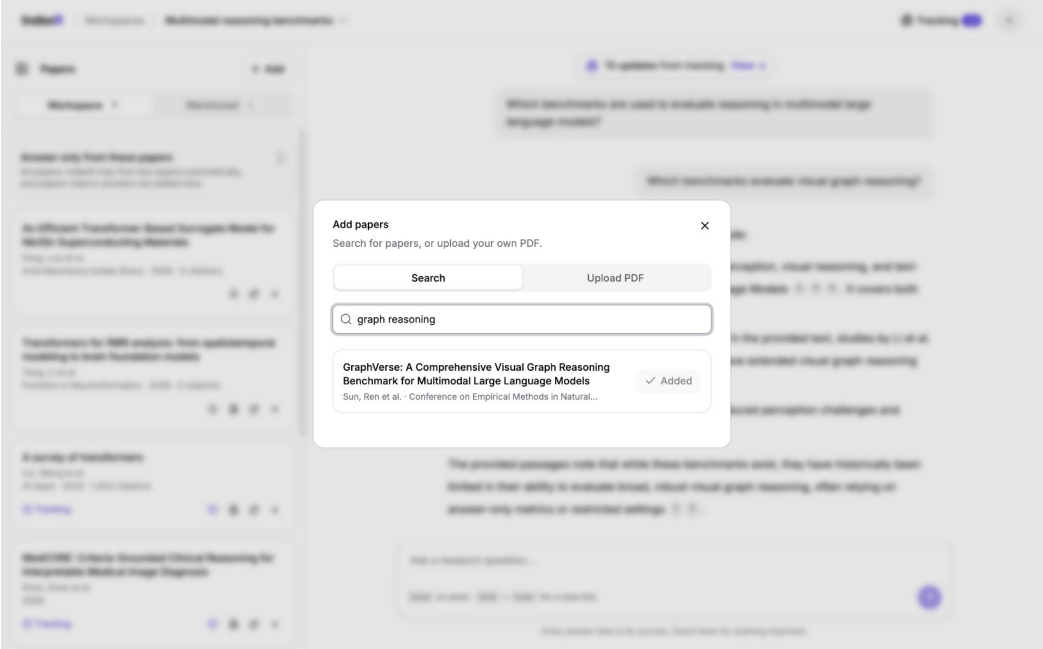


Figure 6.5: Adding a paper from the corpus to a workspace

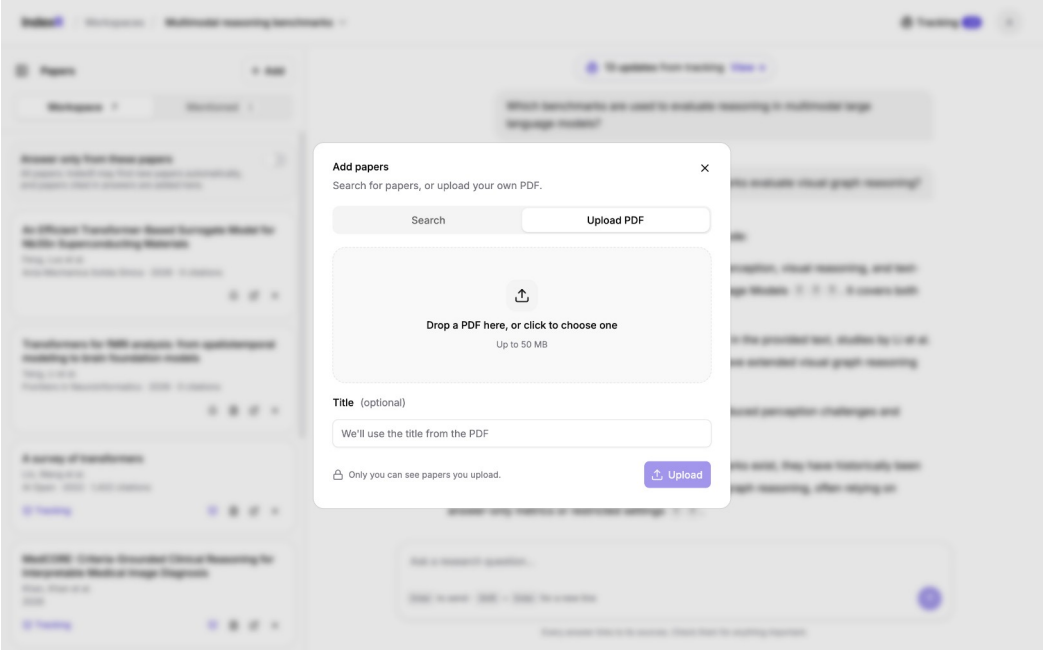


Figure 6.6: Uploading a private PDF

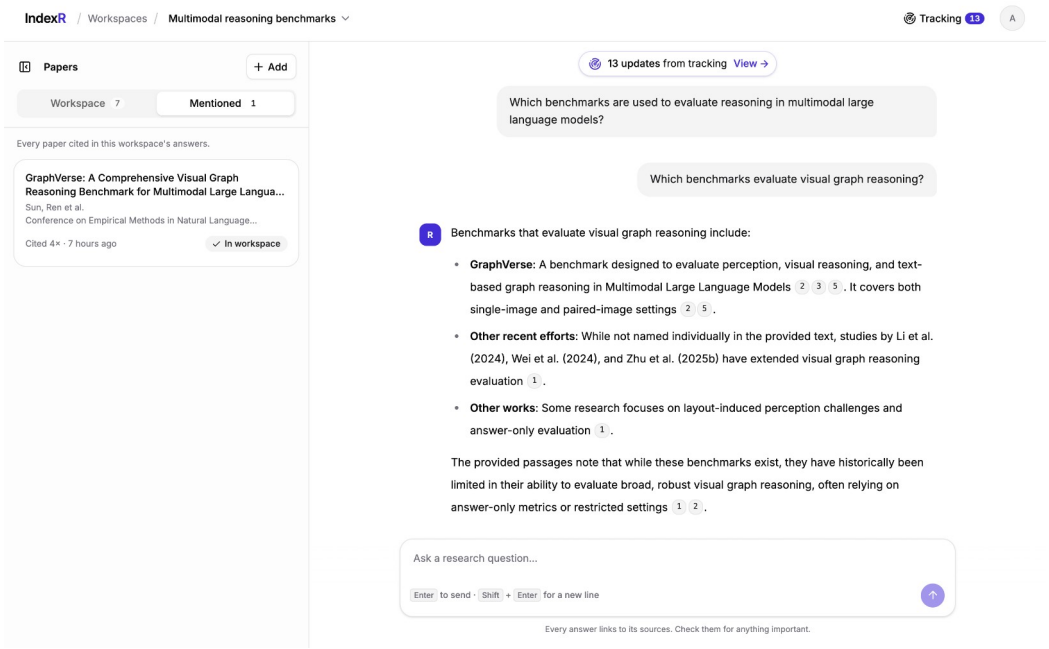


Figure 6.7: Papers mentioned by the workspace's answers

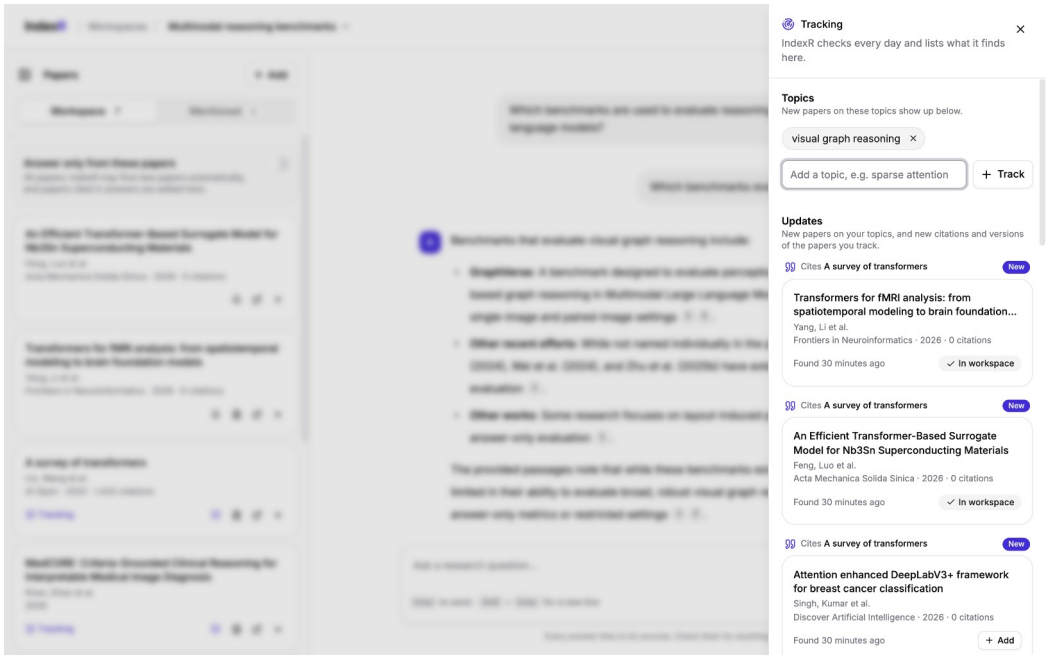


Figure 6.8: Tracking panel with tracked topics and updates (new citations)

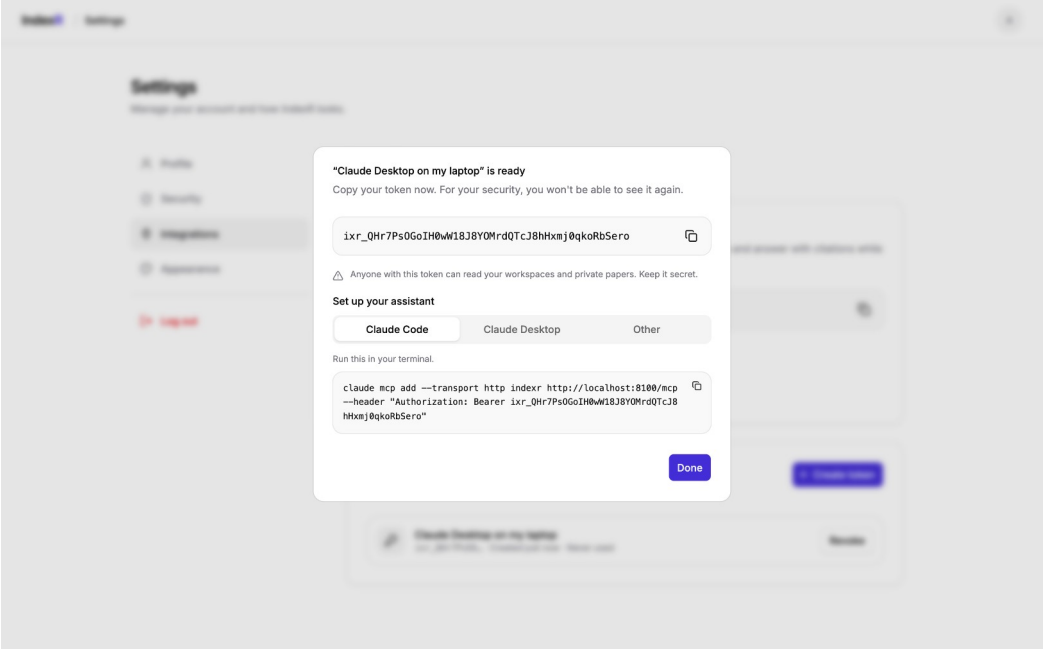


Figure 6.9: Creating an access token with client setup for MCP

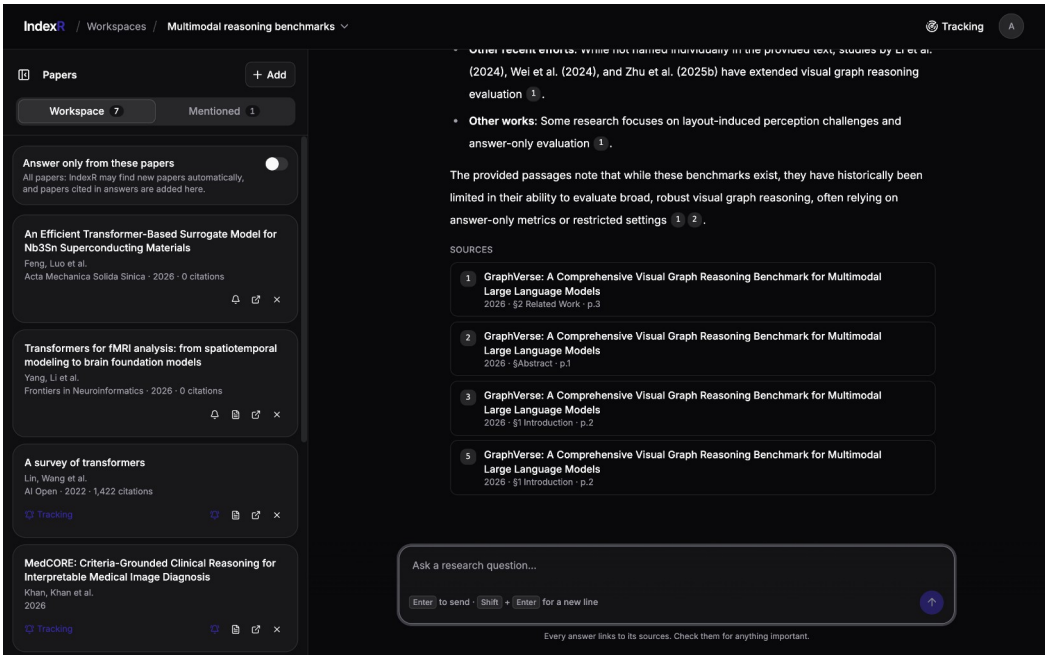


Figure 6.10: Workspace in dark theme

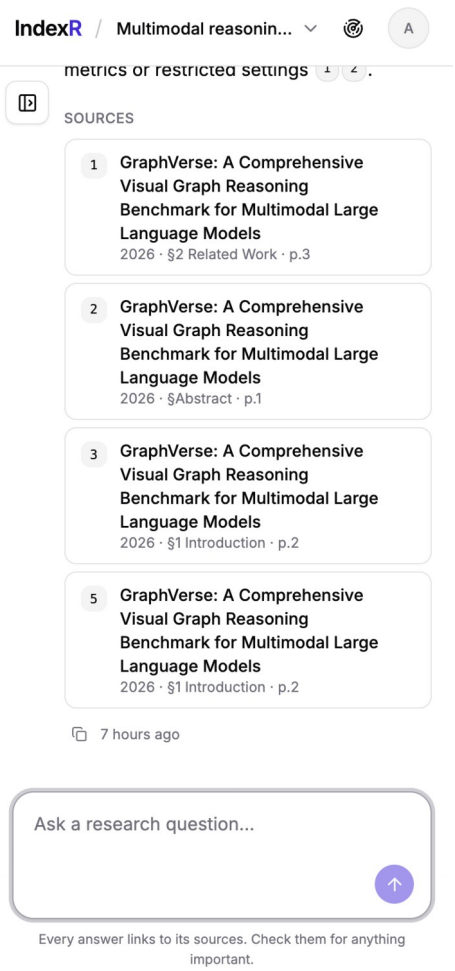


Figure 6.11: Workspace on a mobile screen

[Screenshot placeholder: an AI assistant such as Claude Desktop or Claude Code calling IndexR's ask or search_papers tool]

Figure 6.12: IndexR tools used from an AI assistant (Claude Desktop) through MCP

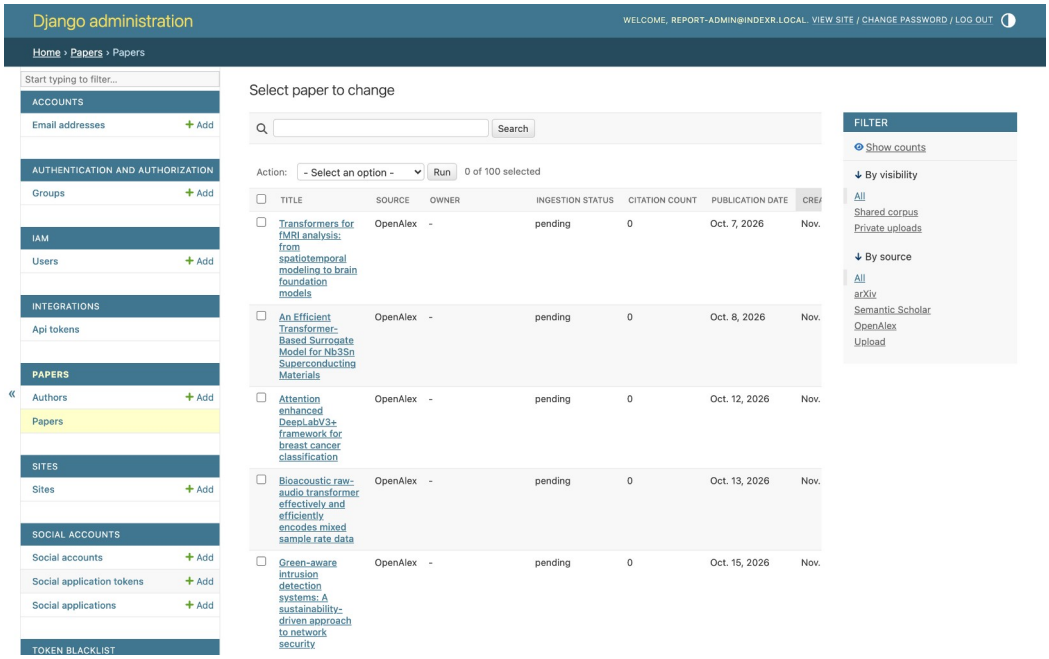


Figure 6.13: Django admin: moderating the shared corpus

6.3 Requirement Verification

Each functional requirement was verified at the layer that enforces it: database constraints with integrity tests, input rules with HTTP 400 tests, behaviour with API and service tests, and the full pipeline with live runs. Table 6.2 summarises the result.

Table 6.2: Requirement verification matrix

ID	Requirement	Verified by	Result
FR1	Register, verify email, log in (password or Google)	IAM API tests; browser checks	Pass
FR2	Create, rename, search and delete own workspaces only	API tests incl. other-user 404s	Pass
FR3	Discover papers from three sources without duplicates	Fetcher, orchestrator, store tests; live run	Pass
FR4	Ingest PDFs into section/page chunks	Extractor and ingestion task tests	Pass
FR5	Rank with full-text search, boost and rerank	Queryset and retrieval tests	Pass
FR6	Discover only when the two-gate rule fires	Retrieval and ask service tests	Pass
FR7	Answer with [n] citations mapped to stored chunks	Answer and ask tests; live answers	Pass
FR8	Stream progress and answer via SSE	Stream view tests; browser checks	Pass
FR9	Private uploads invisible to other users	Visibility tests across search, ask, MCP	Pass
FR10	MCP tools act only as the token’s user	Protocol tests; HTTP 401 checks	Pass

ID	Requirement	Verified by	Result
FR11	Track topics and papers; report each update once	Tracking service, task and API tests; live run	Pass
FR12	Constant query count for paginated lists	Query-count tests (1 vs 5 rows)	Pass

6.4 Automated Testing

The test suite contains 416 tests, organised so that each test file mirrors one source file (Table 6.3). External services (LLM, reranker, paper sources) are replaced with test doubles in unit tests, so the suite is fast and deterministic; the in-memory MCP client of the official SDK is used to test the MCP server through the real protocol. The full suite runs in under 30 seconds, and every run is also checked for warnings, such as the warning Django raises for paginating an unordered list.

Table 6.3: Automated tests by application

Application	Tests	Main areas covered
papers	199	Models and constraints, fetchers, discovery, extraction, retrieval, reranking, answers, uploads, ingestion tasks, admin
workspaces	130	Workspaces, papers, messages, streaming, mentioned papers, topics, tracking updates, tracking service and tasks
integrations	37	Access tokens, MCP tools, MCP protocol and authentication
iam	32	Accounts, permissions, scoping, Google connection
server	18	Shared permissions, pagination, renderers, LLM retries
Total	416	All passing

6.5 Performance and Efficiency

Performance was measured on the development machine described in Chapter 5, using a corpus built by the system itself during development. The results are summarised in Table 6.4.

Table 6.4: Measured performance on the development corpus

Measurement	Result
Corpus size	137 papers (59 fully ingested), 4,930 chunks

Measurement	Result
Full-text search, top 50 with boost (24 queries)	Median 2.2 ms, 95th percentile 7.7 ms, maximum 15.5 ms
Answer from the existing corpus (expand, search, rerank, generate)	About 11 s end to end
Paginated list endpoints	Same number of database queries for 1 and 5 rows (tested for every list)
Daily tracking check (1 topic, 3 tracked papers)	About 30 s, dominated by external API calls

The search figures confirm that the GIN-indexed full-text search is not a bottleneck: almost all of the answer latency is spent in the external LLM and reranker calls, which run once per question. This matches the design goal of roughly 8–12 seconds per answer on a warm workspace.

6.6 Live Functional Validation

Besides automated tests, every phase was validated against the real external services and the running application (Table 6.5). These runs also found two defects that the unit tests had missed, both of which were fixed and covered by new regression tests.

Table 6.5: Live validation runs

Run	Observation
Ask over MCP (stdio)	Answer returned in 11 s with progress updates and four citations to section and page
MCP over HTTP	Requests without a token or with a wrong token rejected with HTTP 401; valid token listed tools, workspaces and search results
Topic tracking	“visual graph reasoning” found 3 new papers from October 2026; an immediate re-run found none (no duplicates)
Citation tracking	A survey with 1,422 citations returned its 10 newest citing papers, stored as citation edges; re-run created no duplicates
Version tracking	Revision date of a tracked arXiv paper recorded as a baseline for later version updates
Defect found: arXiv batches	One malformed arXiv ID made arXiv reject a whole batch; invalid IDs are now filtered out
Defect found: workspace paper update	Updating a workspace paper could replace the paper itself; the paper ID is now read-only on update
UI checks (Playwright)	Core flows in light/dark themes at desktop and mobile widths; a mobile layout issue found and fixed

6.7 Validation Metrics for Evaluation

The metrics in Table 6.6 will be measured on a fixed set of 20–30 research questions with manually judged evidence during Major Project-II.

Table 6.6: Validation metrics and targets

Metric	Definition	Target
Retrieval precision	Questions whose correct evidence is in the top 5 passages	≥ 80%
Citation accuracy	Cited section and page actually contain the cited claim (manual spot-check)	No invented locations
Discovery relevance	Automatically discovered papers judged relevant	≥ 70%
Tracking correctness	Updates that match manual verification; false updates	No false updates
Answer latency	Time from question to complete answer on a warm workspace	≈ 8–12 s
Search latency	Median full-text search time	< 50 ms

6.8 Discussion and Limitations

The outcomes show that a complete RAG system with fine-grained citations can be built without a vector database, and that lexical search is fast enough that the external model calls dominate latency. The current system has known limitations: only open-access PDFs can be read; tables and figures are not extracted; when a tracked arXiv paper gets a new version, the update is reported but the paper's text is not re-read, to protect the passages that earlier answers cite; papers known only to Semantic Scholar without a DOI cannot be checked for citations; and the retrieval and citation metrics above have not yet been measured on a formal evaluation set.

CHAPTER 7
TIMELINE

The project followed the 14-week milestone plan of the Major Project-I course for the August–December 2026 session. Table 7.1 lists the weekly plan and its milestones, and Fig. 7.1 shows it as a Gantt chart. Implementation ran alongside design and validation in nine short phases, each merged after review.

Table 7.1: Weekly plan with milestones

Week	Milestone	Work carried out
1–2	Problem identification	Defined the problem, objectives and deliverables; chose the vectorless approach
3–4	Literature survey	Reviewed retrieval, reranking, RAG, citation and scholarly-data literature; identified the research gap
5–6	Feasibility study	Prepared the block diagram; selected Django, PostgreSQL full-text search, Celery, React, the paper sources, LLM and reranker
7–8	System design	Architecture, data-flow diagrams, flowcharts and database design; phase 1: models, search, LLM interface, ingestion
9–10	Design validation	Phases 2–5: boost and filters, query expansion, reranking, discovery, grounded answers, streaming; requirement verification and testing
9–10	Implementation (cont.)	Phase 6: web application; phase 7: private uploads and admin
11–12	Report drafting	Phases 8–9: MCP server and live tracking; performance and live validation; interim report
13–14	Presentation and viva	Internal review by the project committee

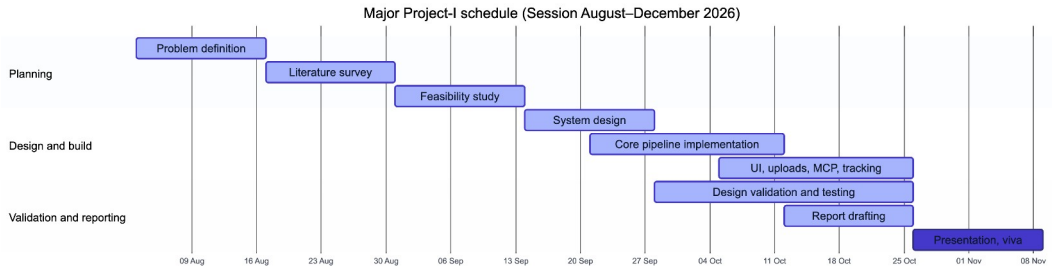


Figure 7.1: Gantt chart of the project schedule

CHAPTER 8
CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project set out to build a research assistant that finds the right papers for a question, answers with citations traceable to the exact section and page, keeps each topic up to date, and does so without a vector database. IndexR meets these objectives. It discovers papers automatically from three open sources, ingests them once into a shared corpus of section- and page-aware chunks, retrieves with PostgreSQL full-text search boosted by citations and recency, broadens queries with an LLM, refines results with a cross-encoder, and decides by a simple two-gate rule when it must look outside its corpus. Its answers cite numbered passages whose locations come from the database, not from the model.

Around this core, the project delivered a complete product: a responsive web application, private uploads visible only to their owner, an MCP server that brings the corpus into AI assistants, and live tracking of new papers, citations and versions. The implementation is backed by 416 automated tests and by live runs that confirmed the behaviour against the real services and exposed two defects that were then fixed. The results support the project's central claim: for research text full of exact terms, lexical retrieval with LLM query expansion and reranking is a practical, inexpensive and explainable foundation for trustworthy, cited answers.

8.2 Future Scope

- **Formal evaluation.** Build the 20–30 question evaluation set and measure retrieval precision, citation accuracy and discovery relevance, then tune the gate thresholds and boost weights on it.
- **Citation-graph expansion.** Use the stored citation edges to add highly connected papers to the candidate set and to visualise how papers in a workspace relate.
- **Richer extraction.** Extract tables, figures and equations so that numerical results can be cited directly.

- **Version-aware re-ingestion.** Re-read new arXiv versions while keeping the passages that earlier answers cite.
- **Notifications and digests.** Email digests and short LLM summaries of what changed in a topic.
- **Collaboration.** Shared workspaces and organisation accounts for research groups and departments.
- **Admin review queue.** Let users request specific papers, with administrator approval before they enter the shared corpus.

REFERENCES

[1] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[3] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[4] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proc. 43rd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2020, pp. 39–48.

[5] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Proc. Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[6] R. Nogueira and K. Cho, "Passage re-ranking with BERT," *arXiv preprint* arXiv:1901.04085, 2019.

[7] J. Lin, R. Nogueira, and A. Yates, *Pretrained Transformers for Text Ranking: BERT and Beyond*, ser. Synthesis Lectures on Human Language Technologies. Morgan & Claypool, 2021.

[8] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[9] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 6465–6488.

[10] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[11] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint* arXiv:2312.10997, 2023.

[12] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[13] L. Gao, X. Ma, J. Lin, and J. Callan, “Precise zero-shot dense retrieval without relevance labels,” in *Proc. 61st Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2023, pp. 1762–1777.

[14] L. Wang, N. Yang, and F. Wei, “Query2doc: Query expansion with large language models,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 9414–9423.

[15] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, “Query rewriting in retrieval-augmented large language models,” in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 5303–5315.

[16] J. Priem, H. Piwowar, and R. Orr, “OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts,” *arXiv preprint* arXiv:2205.01833, 2022.

[17] R. Kinney *et al.*, “The Semantic Scholar open data platform,” *arXiv preprint* arXiv:2301.10140, 2023.

[18] K. Lo, L. L. Wang, M. Neumann, R. Kinney, and D. S. Weld, “S2ORC: The Semantic Scholar open research corpus,” in *Proc. 58th Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 4969–4983.

[19] R. Nakano *et al.*, “WebGPT: Browser-assisted question-answering with human feedback,” *arXiv preprint* arXiv:2112.09332, 2021.

[20] B. Bohnet *et al.*, “Attributed question answering: Evaluation and modeling for attributed large language models,” *arXiv preprint* arXiv:2212.08037, 2022.

[21] A. Cohan, S. Feldman, I. Beltagy, D. Downey, and D. S. Weld, “SPECTER: Document-level representation learning using citation-informed transformers,” in *Proc. 58th Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 2270–2282.

[22] Anthropic, “Model Context Protocol specification.” [Online]. Available: https://modelcontextprotocol.io/specification

[23] PostgreSQL Global Development Group, “PostgreSQL documentation: Full text search.” [Online]. Available: https://www.postgresql.org/docs/current/textsearch.html

[24] Cohere, “Rerank,” Cohere documentation. [Online]. Available: https://docs.cohere.com/docs/rerank-overview

[25] arXiv, “arXiv API user’s manual.” [Online]. Available: https://info.arxiv.org/help/api/user-manual.html

[26] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[27] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.